

Processador SAP-1

Relatório Final

Projeto e implementação em Verilog · Verificação por simulação

*Plataforma-alvo: Terasic DE10-Lite (Intel MAX 10 10M50DAF484C7G) · Quartus
Prime Lite 20.1*

Disciplina: _____

Professor(a): _____

Integrantes:

Diogo Loreiro Campos

Ivan Marcos

Marjorie Lobo

Data: ____ / ____ / ____

Sumário

Para inserir o índice automático no Word: posicione o cursor aqui e use Referências ► Sumário ► Sumário Automático. Os títulos deste documento usam os estilos Título 1/2, então o índice é gerado e numerado automaticamente.

Resumo

Este trabalho apresenta o projeto, a implementação em Verilog e a verificação do processador didático SAP-1 (Simple-As-Possible), com execução na placa FPGA Terasic DE10-Lite. O processador — de 8 bits, com barramento único, memória de 16 palavras e cinco instruções — foi descrito de forma modular (um arquivo por bloco), tendo a unidade de controle modelada como máquina de estados que gera uma palavra de controle de 12 bits. A verificação foi feita por simulação no ModelSim, com 14 testbenches auto-verificáveis (por componente e do sistema completo) e análise de formas de onda. Quatro programas de teste — potência, uma expressão aritmética, multiplicação e divisão — produziram os resultados esperados (27, 18, 12 e 3) e o processador parou corretamente em todos. O resultado é um núcleo sintetizável, determinístico na partida e documentado, que roda de verdade na placa.

Palavras-chave: SAP-1; Verilog; FPGA; DE10-Lite; máquina de estados; simulação.

Objetivos

Objetivo geral:

Implementar e validar o processador SAP-1 em Verilog, do projeto lógico até a execução na FPGA DE10-Lite, compreendendo na prática o funcionamento interno de um processador.

Objetivos específicos:

- Compreender a arquitetura do SAP-1 (datapath, barramento e unidade de controle).
- Descrever cada bloco como um módulo Verilog, com boas práticas de projeto para FPGA.
- Modelar a unidade de controle como uma máquina de estados que gera a palavra de controle.
- Verificar o comportamento por simulação — componente a componente e o sistema completo.
- Executar programas reais na placa DE10-Lite e observar o funcionamento nos displays.
- Documentar o projeto, as decisões de implementação e os resultados.

Metodologia

O trabalho seguiu uma abordagem incremental e verificável, em quatro etapas:

- **Projeto modular:** cada bloco do datapath e o controle viraram arquivos Verilog separados, integrados por um módulo de topo através do barramento.
- **Controle por máquina de estados:** a sequência de microoperações foi centralizada em um contador de anel (T1-T6) que emite a palavra de controle.
- **Verificação por simulação:** antes de ir à placa, cada módulo foi exercitado por um testbench auto-verificável e o sistema completo foi testado com programas reais, analisando as formas de onda no ModelSim.
- **Execução na FPGA:** o núcleo, sem alterações, foi adaptado à DE10-Lite (divisor de clock, botão com antirruído, displays) e gravado para execução real.

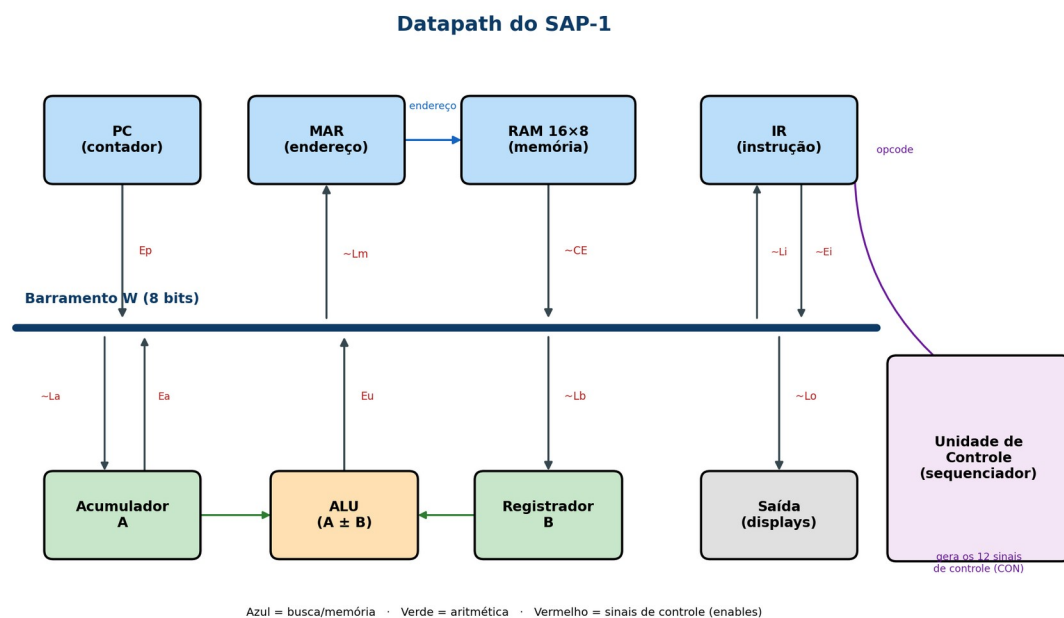
Ferramentas utilizadas: Quartus Prime Lite 20.1 (síntese e gravação), ModelSim ASE (simulação), Icarus Verilog (referência cruzada) e Python/matplotlib (diagramas). A descrição detalhada da verificação está na Parte II.

Fundamentação teórica

Esta seção reúne os conceitos que embasam o projeto, antes das partes de implementação e verificação. O objetivo é tornar o relatório autocontido: quem não conhece o SAP-1 encontra aqui o vocabulário necessário para acompanhar o restante.

O processador SAP-1

O SAP-1 (Simple-As-Possible) é um processador didático proposto por Albert Malvino em Digital Computer Electronics. Sua finalidade não é desempenho, mas ensino: é o menor computador que ainda ilustra todos os princípios de um processador real. Apesar de ter só 8 bits, um único barramento e cinco instruções, ele executa o mesmo ciclo fundamental de qualquer CPU — buscar uma instrução na memória, decodificá-la e executá-la — repetidamente, até parar.



Visão geral do datapath do SAP-1 (detalhado na Parte I).

Arquitetura de Von Neumann

O SAP-1 adota a arquitetura de Von Neumann: instruções e dados residem na mesma memória e trafegam pelo mesmo barramento. A principal consequência prática, explorada ao longo do relatório, é que os 16 endereços da memória são compartilhados entre o programa e os dados — um programa maior deixa menos espaço para dados. Essa unificação simplifica o hardware (uma só memória, um só barramento) ao custo de limitar o tamanho dos programas.

Ciclo de instrução: busca, decodificação e execução

Toda instrução passa por três fases conceituais. Na busca, o endereço da instrução (guardado no contador de programa) é enviado à memória e a instrução lida é levada ao registrador de instrução. Na decodificação, o campo de operação (opcode) é interpretado pela unidade de controle. Na execução, a unidade de controle gera a sequência de sinais que realiza a operação (carregar um dado, somar, subtrair, exibir, parar). No SAP-1 essas fases são realizadas em seis passos de tempo (T1-T6): os três primeiros são a busca, iguais para toda instrução, e os três últimos a execução, que depende do opcode.

Conjunto de instruções (ISA)

O SAP-1 possui cinco instruções. Cada palavra de 8 bits divide-se em dois campos de 4 bits: o opcode (o que fazer) e o operando (um endereço de memória). O endereçamento é direto — o operando indica onde está o dado, não o próprio valor.

Instrução	Opcode	Significado
LDA end	0000	carrega o acumulador com o conteúdo de 'end'
ADD end	0001	soma ao acumulador o conteúdo de 'end'
SUB end	0010	subtrai do acumulador o conteúdo de 'end'
OUT	1110	copia o acumulador para o registrador de saída
HLT	1111	para o processador

Não há instruções de escrita em memória, de desvio ou de laço. Isso significa que os programas são sequências lineares e que operações como multiplicação e divisão precisam ser expressas por somas e subtrações repetidas — tema retomado na Parte II.

Unidade de controle e microoperações

A execução de cada instrução se decompõe em microoperações — pequenas transferências entre registradores comandadas por sinais de controle. A unidade de controle do SAP-1 é do tipo cabeada (hardwired): a cada passo de tempo ela emite uma palavra de controle cujos bits habilitam quem escreve e quem lê no barramento. Como essa palavra depende apenas do passo de tempo e do opcode — nunca do dado —, o comportamento é determinístico. A construção dessa unidade como máquina de estados é o núcleo da Parte I.

Representação numérica

Todos os valores são de 8 bits sem sinal explícito (0 a 255). A subtração usa complemento de dois, e tanto somas quanto subtrações "dão a volta" em 256 (aritmética modular) — por exemplo, $200 + 100$ resulta em 44. Esse comportamento é inerente à largura fixa de 8 bits e é verificado em simulação na Parte II.

Parte I — Projeto e implementação

Esta parte descreve a estrutura do projeto, as decisões de implementação, a máquina de estados, a palavra de controle e o código-fonte comentado. A numeração de seções a seguir é interna a esta parte.

1. Introdução

Este relatório documenta a implementação em Verilog do processador SAP-1 (Simple-As-Possible): como o projeto está organizado, quais decisões de projeto foram tomadas — e por quê, no nível de hardware —, como a temporização foi disciplinada em duas bordas de clock, como a unidade de controle foi modelada por máquina de estados, e o código-fonte comentado. Serve de referência técnica do que foi construído, complementando o relatório de verificação por simulação.

Resumo das especificações implementadas:

Parâmetro	Valor	Onde
Largura de dados	8 bits	barramento W, A, B, ULA, saída
Largura de endereço	4 bits	PC e MAR (16 posições)
Memória	16 × 8 bits (ROM em execução)	ram_16x8.v
Registradores de dados	2 (acumulador A e B)	accumulator.v, register_b.v
ULA	soma e subtração, 8 bits	adder_subtractor.v
Instruções	5 (LDA, ADD, SUB, OUT, HLT)	decodificadas no controle
Estados de controle	6 (T1-T6) + IDLE de partida	controller_sequencer.v
Palavra de controle	12 bits (CON)	gerada a cada estado

2. Estrutura do projeto

O projeto separa o núcleo sintetizável (rtl) da camada específica de placa (fpga), da verificação (tb_model) e da documentação:

```
SAP1/  
|- rtl/          nucleo do processador (11 modulos, sintetizaveis, sem I/O de placa)  
|- fpga/         camada da placa DE10-Lite (top de placa, clock, debounce, displays)  
|- tb_model/     14 testbenches auto-verificaveis + scripts de onda (ModelSim)  
|- programas/   programas de teste em .txt (1..4) para colar na RAM  
|- fsm/         diagramas da maquina de estados (Python/matplotlib)
```


- docs/ documentacao e relatorios (este arquivo)
- apresentacao/ slides e imagens (datapath, ondas)
- sap1_top.qsf, SAP1.qpf projeto Quartus (pinos, arquivos)

A separação **rtl** x **fpga** é deliberada e tem efeito prático: o núcleo em **rtl** não conhece displays, botões nem o clock de 50 MHz da placa — suas únicas entradas são **clk** e **clr_bar**. Quem adapta o núcleo ao hardware real (divide o clock, filtra o botão, aciona os sete segmentos) é a pasta **fpga**. Assim, o mesmo núcleo é simulado e sintetizado sem alteração.

2.1. Módulos do núcleo (rtl)

Módulo	Papel	Natureza
sap1_top.v	Topo: barramento W (mux) + instâncias de todos os blocos	estrutural
program_counter.v	PC — contador de 4 bits do endereço de instrução	sequencial
mar.v	Registrador de endereço apresentado à memória	sequencial
ram_16x8.v	Memória 16×8, leitura combinacional (ROM em execução)	combinacional
instruction_register.v	IR — guarda a instrução e a fatia em opcode operando	sequencial
accumulator.v	Acumulador A (registrador de trabalho)	sequencial
register_b.v	Registrador B (segundo operando da ULA)	sequencial
adder_subtractor.v	ULA — soma/subtração em 8 bits (complemento de dois)	combinacional
output_register.v	Registrador de saída (visor)	sequencial
controller_sequencer.v	Controle usado: anel one-hot + palavra de controle	sequencial
controller_fsm.v	Controle alternativo: FSM clássica de 3 processos	sequencial

2.2. Módulos da camada de placa (fpga)

Módulo	Papel
sap1_fpga.v	Top de placa: junta núcleo, clock, reset, botões e displays
clock_divider.v	Divide 50 MHz para ~1 Hz (execução visível)

debouncer.v	Antirruído do botão (clock manual passo a passo)
seg7.v	Decodificador hexadecimal → 7 segmentos
seg7_instr.v	Decodificador opcode → letra (L/A/5/o/H)

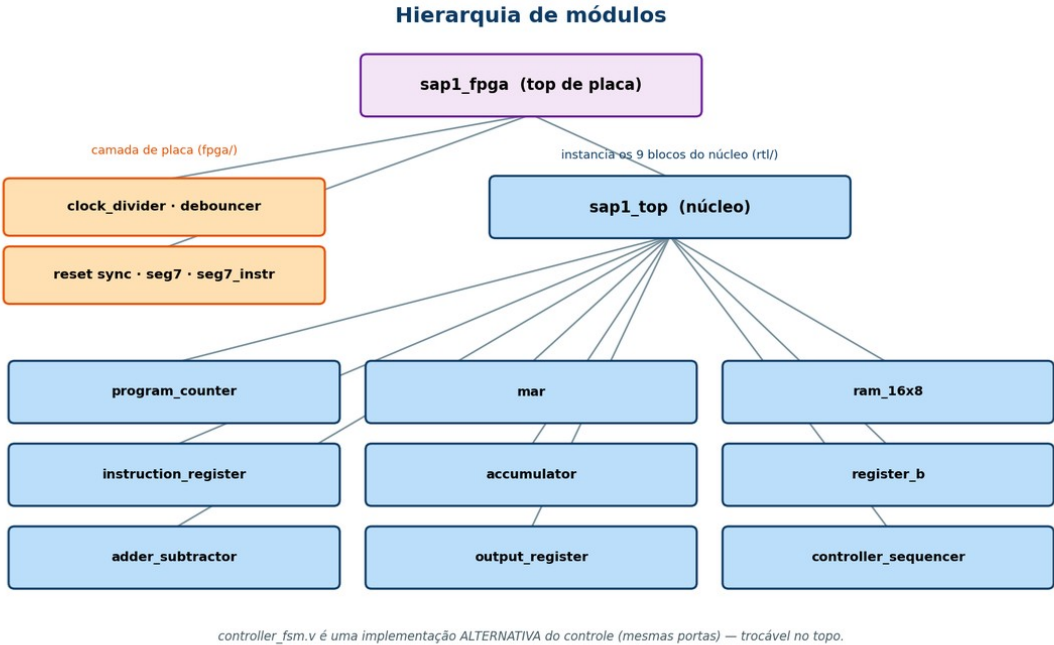


Figura 1 — Hierarquia: o top de placa instancia a camada fpga e o núcleo sap1_top, que por sua vez instancia os 9 blocos do datapath e controle.

A figura evidencia a separação de responsabilidades: o núcleo sap1_top (azul) apenas instancia os nove blocos do datapath e do controle, sem conhecer nada da placa; toda a adaptação ao hardware (clock, botão, displays) fica na camada fpga (laranja), sob o top de placa. Note ainda que controller_fsm.v é uma alternativa ao controller_sequencer.v — mesmas portas, trocável no topo.

3. Arquitetura e convenções de sinais

Datapath do SAP-1

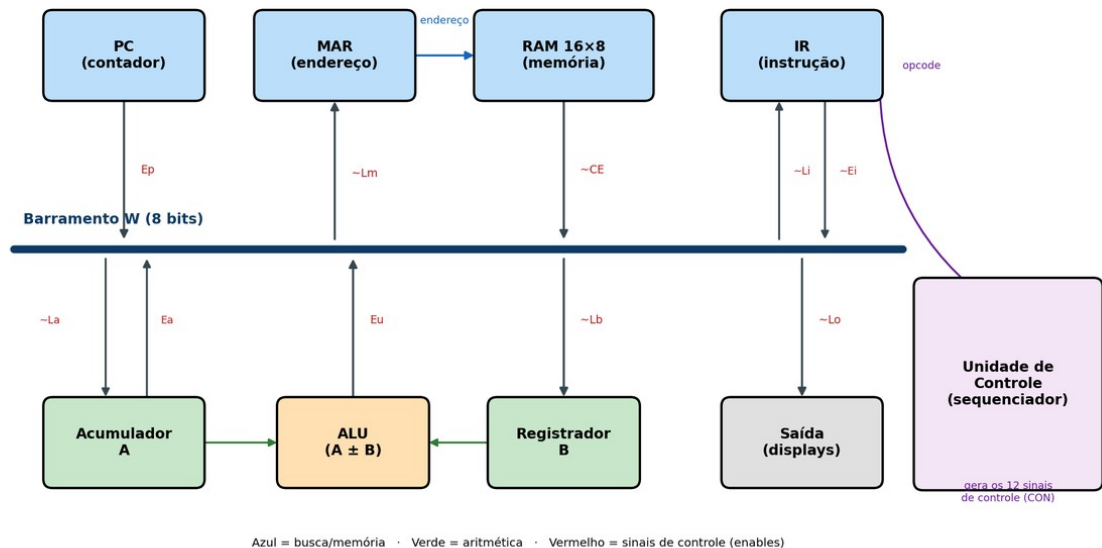


Figura 2 — Datapath do SAP-1: barramento W, blocos de dados e unidade de controle.

Todos os blocos se comunicam por um único barramento de 8 bits (barramento W). A regra de uso do barramento é rígida e é o que garante a coerência do datapath:

- **Em cada estado, no máximo um bloco escreve** no barramento (um único "enable" de saída ativo); um ou mais blocos podem **ler** simultaneamente.
- Fontes de 4 bits (PC e operando do IR) são **estendidas com zeros** à esquerda para 8 bits: {4'b0000, pc_out}. O MAR, por sua vez, lê apenas os 4 bits baixos do barramento (w_bus[3:0]).

3.1. Convenção de nomes e níveis ativos

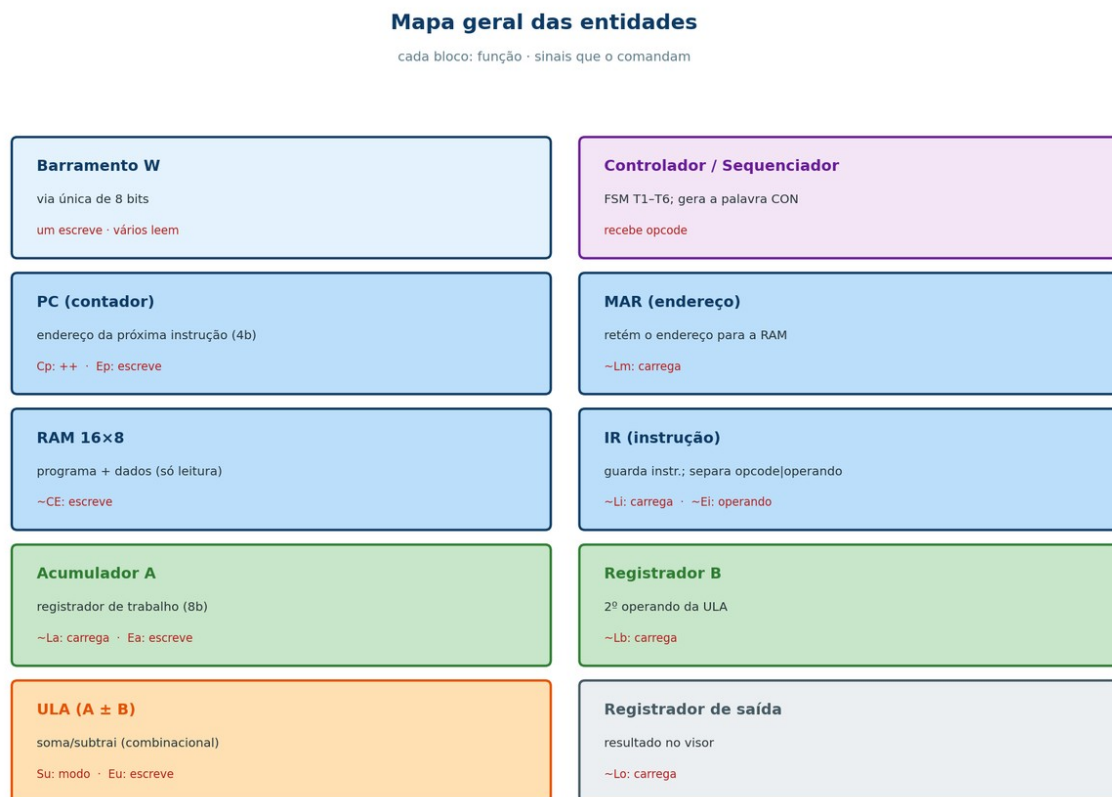
A nomenclatura segue a tradição do SAP-1 e é seguida à risca no código, o que evita erros de polaridade:

- Sinais **ativos em alto** (1 = ativo): Cp, Ep, Ea, Su, Eu. São "habilitações" de contagem/saída no barramento.
- Sinais **ativos em baixo** (0 = ativo), grafados com til/sufixo _bar: ~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo. São as cargas de registradores e a habilitação da RAM.

Consequência importante: o **repouso não é tudo-zero**. Como as cargas são ativas em baixo, a palavra de repouso precisa manter esses bits em 1. Por isso a palavra ociosa é **12'b0011_1110_0011**, e não zero.

3.2. Mapa geral das entidades

Antes de detalhar a temporização e o código, esta é a visão de conjunto: o que cada bloco faz, o que ele recebe e o que entrega. Todos os registradores compartilham o mesmo padrão (carga ativa em baixo, reset assíncrono), variando apenas a largura e o nome do sinal de carga.



Azul = busca/memória · Verde = aritmética · Laranja = ULA · Roxo = controle · Cinza = saída. Vermelho = sinais de controle.

Figura 3 — Mapa geral das entidades: função e sinais de comando de cada bloco.

- **Barramento W (8 bits)** — a via única por onde os dados circulam. Em cada passo, um bloco escreve nele (habilitado por um sinal de saída) e um ou mais leem. É implementado por multiplexador (não tri-state).
- **Contador de Programa (PC)** — guarda o endereço da próxima instrução (4 bits). Recebe: Cp (incrementa) e Ep (coloca o endereço no barramento). Entrega: o endereço corrente. É quem faz o programa avançar.
- **Registrador de Endereço (MAR)** — retém o endereço que a memória vai usar. Recebe do barramento (4 bits) quando ~Lm ativa; entrega o endereço à RAM. Carregado duas vezes por instrução (no T1 e no T4).
- **Memória RAM 16×8** — guarda programa e dados (16 palavras de 8 bits). Endereçada pelo MAR; entrega o conteúdo no barramento quando ~CE ativa. Somente leitura em execução (ROM inicializada).

- **Registrador de Instrução (IR)** — captura a instrução vinda da memória ($\sim Li$) e a separa em opcode (bits altos, para o controle) e operando (bits baixos, que vai ao barramento quando $\sim Ei$ ativa).
- **Acumulador (A)** — registrador de trabalho de 8 bits: acumula os resultados. Recebe do barramento ($\sim La$) e o entrega tanto continuamente à ULA quanto ao barramento (Ea). É o registrador mais movimentado do processador.
- **Registrador B** — guarda o segundo operando da ULA ($\sim Lb$). Diferente de A, serve apenas de "copo" temporário durante somas e subtrações.
- **ULA (somador/subtrator)** — bloco combinacional: calcula $A+B$ ou $A-B$ (conforme Su) e coloca o resultado no barramento quando Eu ativa. Não guarda estado.
- **Registrador de Saída** — recebe o acumulador ($\sim Lo$) e o mantém estável para o visor (LEDs/displays), sem afetar o restante do datapath.
- **Controlador/Sequenciador** — o "maestro": a máquina de estados que percorre T1-T6 e, a cada passo, emite a palavra de controle de 12 bits que aciona todos os sinais acima. Recebe o opcode do IR e decide a sequência de microoperações.

4. Disciplina de temporização em duas bordas

A decisão de temporização mais estrutural do projeto é usar as duas bordas do clock, com papéis distintos:

- O **contador de estados avança na borda de descida** (negedge) do clock.
- Os **registradores do datapath carregam na borda de subida** (posedge).

O motivo é eliminar uma condição de corrida entre "decidir o que fazer" e "fazer". Ao trocar de estado no negedge, a nova palavra de controle já está estável e propagada quando chega o posedge seguinte — instante em que os registradores amostram o barramento. Se estado e dados mudassem na mesma borda, o registrador poderia amostrar valores da microoperação errada. Na prática, cada estado T_n ocupa meio período "preparando" o barramento e o posedge no meio dele efetiva a transferência. Essa relação é o que torna previsível a leitura das ondas (o acumulador de um ADD só assenta no T6 e aparece assentado já no T1 seguinte).

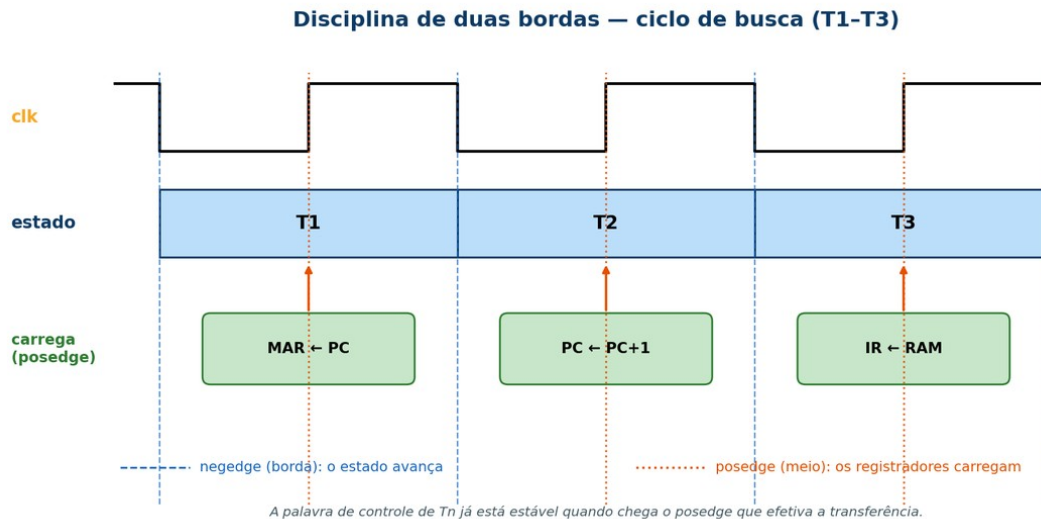


Figura 4 — Duas bordas: o estado avança no negedge (bordas) e os registradores carregam no posedge (meio de cada estado).

A figura resume essa disciplina para o ciclo de busca. Nas linhas azuis (bordas de descida) o estado muda — T1, T2, T3 —; nas linhas laranjas (bordas de subida, no meio de cada estado) os registradores amostram o barramento: $MAR \leftarrow PC$ em T1, $PC \leftarrow PC+1$ em T2 e $IR \leftarrow RAM$ em T3. Como a palavra de controle de cada estado já está estável antes do posedge correspondente, não há ambiguidade sobre qual transferência ocorre.

5. Decisões de implementação

As decisões a seguir são ilustradas com trechos de código. As construções de Verilog empregadas (*assign*, *always*, *reg/wire*, *reset assíncrono*, *non-blocking*, etc.) estão resumidas na seção 10.1 — vale consultá-la antes se o Verilog não for familiar.

5.1. Barramento W por multiplexador com prioridade

Nos livros o barramento costuma ser desenhado com buffers tri-state. Optou-se por um multiplexador com cadeia de prioridade (if-else aninhado via operador ternário), pela razão concreta de que FPGAs da família MAX 10 não têm barramentos tri-state internos: a forma sintetizável de "vários blocos, um fio" é justamente o mux. A cadeia testa os enables em ordem; funcionalmente a ordem é irrelevante porque a unidade de controle garante exclusão mútua (só um enable ativo por estado). O ramo final (0000_0000) evita propagação de X na simulação quando, por construção, nenhum enable está ativo (estados de repouso).

rtl/sap1_top.v — multiplexador do barramento W

```
assign w_bus = ep          ? {4'b0000, pc_out}      : // Ep : PC (4b) -> 8b
                  (ce_bar==1'b0) ? ram_data          : // ~CE: RAM (8b)
```

```

(ei_bar==1'b0) ? {4'b0000, ir_operand} : // ~Ei: operando (4b) -> 8b
ea      ? acc_out      : // Ea : acumulador
eu      ? alu_out      : // Eu : saída da ULA
      8'b0000_0000;      // repouso (nenhum fala)

```

5.2. Padrão de registrador: reset assíncrono + carga com prioridade

Cinco blocos (A, B, MAR, IR e saída) são o mesmo flip-flop com enable. O reset aparece na lista de sensibilidade (negedge clr_bar), tornando-o assíncrono — ele vence qualquer coisa. Em seguida, a carga só ocorre quando o sinal está ativo; se não houver carga, o bloco simplesmente não atribui, e a síntese infere o "segura o valor" de um flip-flop com clock-enable (não gera latch, pois todo caminho está sob a borda de clock). Padronizar isso deixou o núcleo uniforme — muda só a largura e o nome da carga.

rtl/accumulator.v – padrão de A, B, MAR, IR e saída

```

always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)    acc_out <= 8'b0000_0000; // (1) reset assincrono, prioridade
    else if (!la_bar) acc_out <= bus_in;      // (2) carrega quando ~La = 0
end                                           // (3) senao: mantem (FF com enable)

```

5.3. Memória como ROM inicializada (sem STA)

O SAP-1 básico não tem instrução de escrita (STA); logo a RAM é somente de leitura durante a execução. Um bloco inicial preenche os 16 endereços e a leitura é combinacional (assign data_out = mem[addr]), sem clock — um "asynchronous read ROM". Na síntese isso vira memória de inicialização (ROM). Como programa e dados dividem os mesmos 16 slots (Von Neumann), o tamanho do programa limita o espaço de dados. Trocar de programa = editar este bloco e recompilar; por isso os quatro programas ficam prontos em programas/*.txt.

rtl/ram_16x8.v

```

reg [7:0] mem [0:15];
integer i;
initial begin
    for (i = 0; i < 16; i = i + 1) mem[i] = 8'h00; // zera tudo
    mem[0] = 8'b0000_1011; // LDA 11 <- opcode|operando
    mem[1] = 8'b0001_1100; // ADD 12
    // ... (programa)
    mem[10] = 8'b1111_0000; // HLT
    mem[11] = 8'd7; // (dados)
end
assign data_out = mem[addr]; // leitura assincrona (combinacional)

```

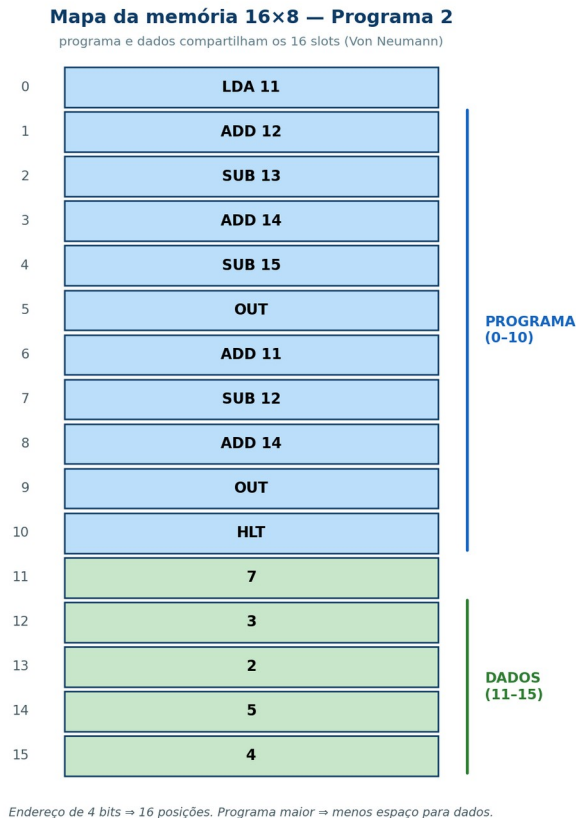


Figura 5 — Os 16 slots compartilhados: programa (0-10) e dados (11-15) do Programa 2.

A figura mostra essa divisão para o Programa 2: as posições 0 a 10 guardam as instruções e 11 a 15 os dados. Fica evidente o compromisso imposto pelos 4 bits de endereço — programa e dados disputam os mesmos 16 slots, de modo que um programa maior deixa menos espaço para dados.

5.4. ULA combinacional em complemento de dois

A ULA é uma única expressão combinacional. A subtração usa o complemento de dois nativo do Verilog e o resultado de 8 bits satura por wraparound (módulo 256). Por ser combinacional, o resultado precisa estabilizar dentro do estado T6 antes do posedge que o grava em A — o que é folgado no clock lento da placa. É essa largura fixa de 8 bits que limita o SAP-1 a valores de 0 a 255.

`rtl/adder_subtractor.v`

```
assign result = su ? (acc - breg) // Su = 1: subtrai (compl. de dois)
                : (acc + breg); // Su = 0: soma
```

5.5. HLT por clock-enable, não por gating de clock

Decisão de robustez central. Parar o processador desligando o clock por lógica combinacional (clock gating) é má prática em FPGA: introduz glitches no sinal de clock, viola as restrições de tempo (STA) e pode deixar registradores em estados inconsistentes. A solução adotada é registrar um

flag halted (travado no estado T4, quando o opcode HLT já foi decodificado em T3) e usá-lo como clock-enable do contador de estados. Congelado o contador, a palavra de controle vira IDLE, nenhuma carga é ativada e todos os registradores seguem seus valores naturalmente — o processador "para" com o clock intacto e o estado preservado em T4. No topo, o clock do núcleo é ligado direto (wire clk_cpu = clk;), sem qualquer gating.

rtl/controller_sequencer.v – HALT registrado

```
reg halted;
wire run = ~halted; // clock-enable do contador de anel
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar) halted <= 1'b0;
    else if ((ring == T4) && (opcode == HLT)) halted <= 1'b1; // trava em T4
end
```

5.6. Estado ocioso de partida (IDLE) e recuperação de reset

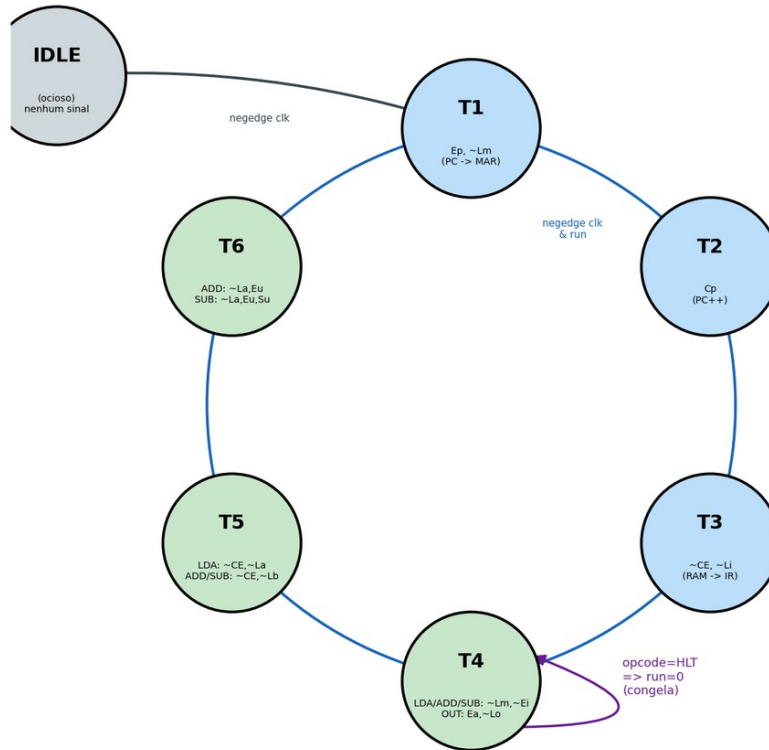
O contador inclui um estado de partida IDLE (nenhum T ativo, valor one-hot 000000). Ele absorve a fase do reset. Sem esse estado, o funcionamento correto dependeria de soltar o reset exatamente com o clock em nível conveniente — um problema clássico de recovery/removal do reset assíncrono. Com o IDLE, qualquer que seja a fase do clock ao soltar o reset, o primeiro negedge útil leva a máquina a T1 de forma determinística. Na camada de placa, esse cuidado é reforçado por um sincronizador de reset (assert assíncrono, desassert síncrono), descrito na seção 9.

6. Máquina de estados (unidade de controle)

6.1. Diagrama de estados (anel)

$\sim\text{CLR}=0$
(reset assinc.)

FSM do Controlador/Sequenciador - SAP-1



Azul = ciclo de BUSCA (T1-T3, igual p/ toda instrução). Verde = ciclo de EXECUÇÃO (T4-T6, depende do opcode).
Estado avança na borda de DESCIDA do clock; registradores carregam na SUBIDA.

Figura 6 — Contador de anel IDLE → T1 → ... → T6 → T1; laço roxo em T4 = congelamento por HLT.

6.2. Ciclo de execução por instrução

Ciclo de Execucao por Instrucao - FSM do SAP-1

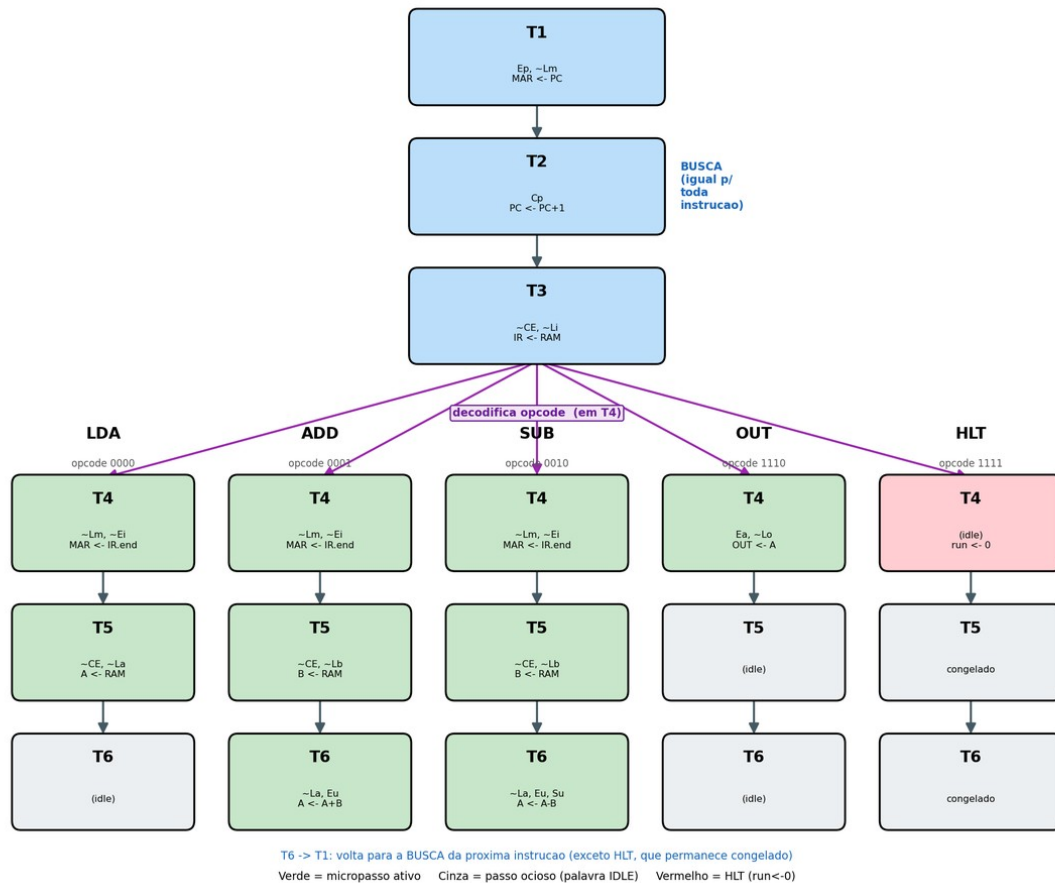


Figura 7 — Busca comum (T1-T3, azul) e ramificação da execução (T4-T6) por opcode.

A busca é idêntica para toda instrução; a diferença começa em T4, quando o opcode é decodificado. LDA usa até T5 ($A \leftarrow RAM$); ADD/SUB usam até T6 ($A \leftarrow A \pm B$); OUT resolve em T4; HLT congela em T4.

6.3. Codificação one-hot e a rotação do anel

O estado é mantido em one-hot de 6 bits (exatamente um bit em 1 por vez, salvo o IDLE que é todo-zero). One-hot foi escolhido por dois motivos concretos: a decodificação do estado vira um simples teste de um bit (barato e rápido em LUTs de FPGA), e a transição para o próximo estado é uma rotação de 1 bit — sem somador nem comparador. A rotação é feita por concatenação:

rotação one-hot (T6 → T1 fecha o anel)

```
// ring_r = {b5,b4,b3,b2,b1,b0} (T1=000001, T2=000010, ..., T6=100000)
ring_r <= {ring_r[4:0], ring_r[5]}; // desloca a esquerda e traz o bit 5 p/ o 0
// T1(000001) -> T2(000010) -> T3(000100) -> T4(001000) -> T5(010000) -> T6(100000) -> T1
```

O contador completo trata os quatro casos — reset, congelado (HLT), partida do IDLE e rotação — nesta ordem de prioridade:

rtl/controller_sequencer.v – contador de anel

```
always @(negedge clk or negedge clr_bar) begin
    if (!clr_bar)        ring_r <= IDLE_ST;           // reset -> IDLE
    else if (!run)        ring_r <= ring_r;           // HLT: congela
    else if (ring_r==IDLE_ST) ring_r <= T1;           // 1o negedge -> T1
    else                  ring_r <= {ring_r[4:0], ring_r[5]}; // rotaciona
end
```

6.4. Duas implementações equivalentes do controle

A unidade de controle existe em dois arquivos, funcionalmente equivalentes (mesmas portas, mesma tabela de saída), como estudo comparativo:

Aspecto	controller_sequencer.v (usado)	controller_fsm.v (alternativo)
Estilo	contador de anel (shift register)	FSM clássica de 3 processos
Codificação do estado	one-hot (6 bits)	binária (3 bits, nomes simbólicos)
Próximo estado	rotação de 1 bit	case explícito S_T1→S_T2→...
Vantagem	decode/rotação triviais	leitura didática, estados nomeados

Trocar uma pela outra é só mudar o nome do módulo instanciado no topo (as portas são idênticas). O trecho de próximo-estado da versão clássica deixa a sequência explícita:

rtl/controller_fsm.v – lógica de próximo estado (2º processo)

```
always @(*) begin
    if (!run) next_state = state;           // HLT: congela
    else case (state)
        S_IDLE: next_state = S_T1;
        S_T1:  next_state = S_T2;   S_T2: next_state = S_T3;
        S_T3:  next_state = S_T4;   S_T4: next_state = S_T5;
        S_T5:  next_state = S_T6;   S_T6: next_state = S_T1; // fecha o anel
        default: next_state = S_IDLE;
    endcase
end
```

6.5. Descrição estado a estado (T1-T6)

A seguir, o papel de cada passo de tempo. Os três primeiros formam a busca e são idênticos para toda instrução; os três últimos formam a execução e dependem do opcode. Em todos, a palavra de controle do estado é preparada no negedge e a transferência se efetiva no posedge seguinte.

T1 — envia o endereço da instrução (MAR ← PC). O contador de programa é habilitado a escrever no barramento (Ep) e o MAR a carregar

($\sim Lm$). Ao fim do estado, o MAR contém o endereço da instrução a buscar. É o passo que "aponta" para a memória.

T2 — incrementa o contador ($PC \leftarrow PC + 1$). Apenas Cp fica ativo: o PC soma 1, passando a apontar para a próxima instrução. O barramento não é usado para transferência — este passo apenas adianta o PC enquanto o endereço atual já está seguro no MAR.

T3 — busca e decodifica ($IR \leftarrow RAM$). A RAM é habilitada a escrever no barramento ($\sim CE$) e o IR a carregar ($\sim Li$). A instrução apontada pelo MAR é lida e guardada no IR, que imediatamente expõe o opcode ao controle. A partir daqui a máquina "sabe" qual é a instrução — a busca terminou.

T4 — primeiro passo da execução (depende do opcode). Para LDA/ADD/SUB, o operando do IR vai ao MAR ($\sim Ei, \sim Lm$), preparando a leitura do dado. Para OUT, o acumulador é copiado ao registrador de saída (Ea, $\sim Lo$) e a instrução termina aqui. Para HLT, o flag de parada é travado e a máquina congela em T4.

T5 — busca o operando (depende do opcode). Para LDA, o dado lido da RAM vai direto ao acumulador ($\sim CE, \sim La$) — a instrução acaba. Para ADD/SUB, o dado vai ao registrador B ($\sim CE, \sim Lb$), ficando pronto para a operação aritmética do passo seguinte. Para OUT/HLT, o passo é vazio.

T6 — conclui a aritmética (só ADD/SUB). A ULA calcula $A+B$ (ADD) ou $A-B$ (SUB, com Su) e o resultado retorna ao acumulador (Eu, $\sim La$). Como A só é reescrito neste posedge, o novo valor aparece assentado já no T1 da instrução seguinte. Para as demais instruções, o passo é vazio.

Depois do T6 (ou do passo em que a instrução termina), o anel volta a T1 e o ciclo recomeça para a próxima instrução — exceto após HLT, em que a máquina permanece congelada em T4.

7. A palavra de controle de 12 bits

A cada estado a unidade de controle emite 12 bits (CON) que comandam todo o datapath. O layout é fixo (MSB $\rightarrow$ LSB):

```
CON = { Cp, Ep,  $\sim Lm$ ,  $\sim CE$ ,  $\sim Li$ ,  $\sim Ei$ ,  $\sim La$ , Ea, Su, Eu,  $\sim Lb$ ,  $\sim Lo$  }  
bit: 11 10 9 8 7 6 5 4 3 2 1 0
```

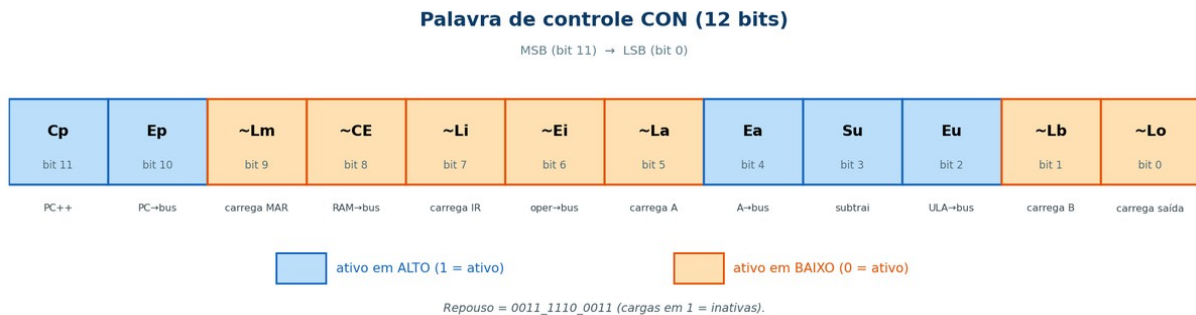


Figura 8 — Layout dos 12 bits de CON: azul = ativo em alto, laranja = ativo em baixo.

Na figura, os bits em azul são ativos em alto (habilitações) e os em laranja, ativos em baixo (cargas e a habilitação da RAM). Essa distinção visual explica por que a palavra de repouso mantém 1 nos bits de carga — em vez de zero — e serve de guia para ler a tabela detalhada a seguir.

Bit	Sinal	Ativo	Função quando ativo
11	Cp	alto	incrementa o PC
10	Ep	alto	PC escreve no barramento
9	~Lm	baixo	carrega o MAR a partir do barramento
8	~CE	baixo	RAM escreve no barramento
7	~Li	baixo	carrega o IR
6	~Ei	baixo	operando do IR no barramento
5	~La	baixo	carrega o acumulador
4	Ea	alto	acumulador escreve no barramento
3	Su	alto	ULA subtrai (0 = soma)
2	Eu	alto	ULA escreve no barramento
1	~Lb	baixo	carrega o registrador B
0	~Lo	baixo	carrega o registrador de saída

A tabela a seguir lista a palavra emitida em cada estado/opcode, os sinais que ficam ativos e a microoperação resultante. Os valores binários foram transcritos diretamente do código (e conferidos bit a bit):

Estado	Opcode	CON (12 bits)	Sinais ativos	Microoperação
IDLE	—	0011_1110_0011	(nenhum)	repouso
T1	todos	0101_1110_0011	Ep, ~Lm	MAR ← PC
T2	todos	1011_1110_0011	Cp	PC ← PC + 1
T3	todos	0010_0110_0011	~CE, ~Li	IR ← RAM[MAR]
T4	LDA/ADD/SUB	0001_1010_0011	~Lm, ~Ei	MAR ← IR.operando
T4	OUT	0011_1111_0010	Ea, ~Lo	saída ← A
T4	HLT	(IDLE)	run ← 0	congela em T4
T5	LDA	0010_1100_0011	~CE, ~La	A ← RAM[MAR]
T5	ADD/SUB	0010_1110_0001	~CE, ~Lb	B ← RAM[MAR]
T6	ADD	0011_1100_0111	~La, Eu	A ← A + B
T6	SUB	0011_1100_1111	~La, Eu, Su	A ← A – B

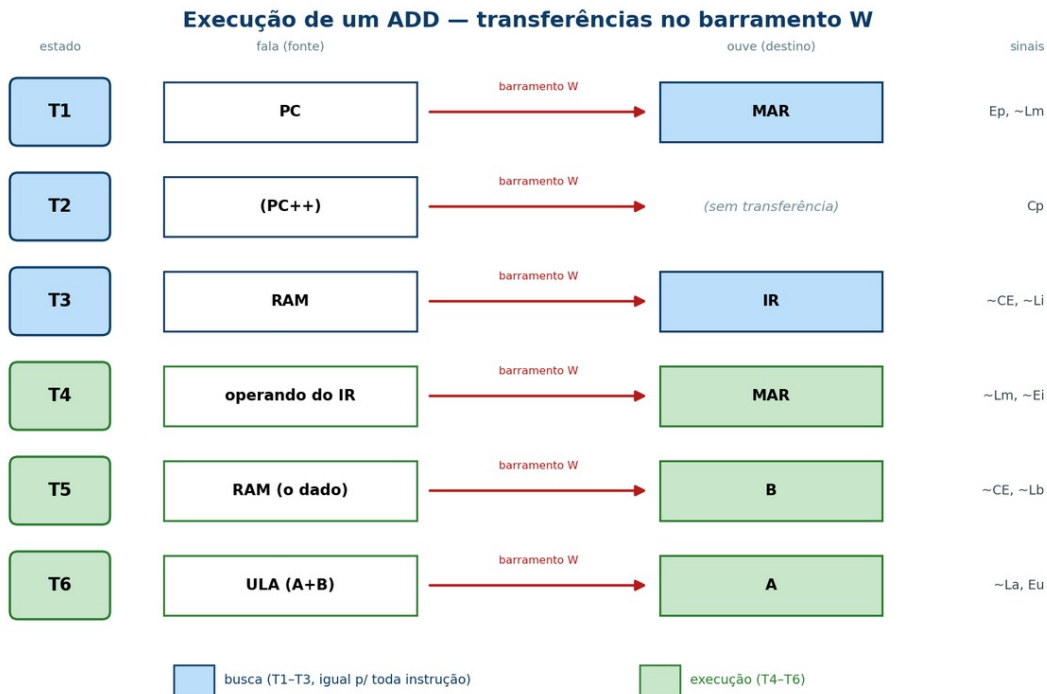
Estados não listados para um dado opcode (por exemplo, T5/T6 de OUT) emitem a palavra IDLE — são "vazios", o que explica por que toda instrução ocupa 6 estados mesmo quando usa menos.

8. Rastreamento ciclo a ciclo de uma instrução (ADD)

Para tornar concreta a interação controle × datapath, segue a execução de um ADD endereço, com o acumulador inicialmente em A e o dado D na posição apontada. Cada linha é um estado: o controle prepara a palavra (no negedge de entrada do estado) e a transferência se efetiva no posedge seguinte.

Estado	Sinais ativos	Quem fala no barramento	Quem carrega (posedge)
T1	Ep, ~Lm	PC	MAR ← PC
T2	Cp	(ninguém)	PC ← PC + 1
T3	~CE, ~Li	RAM	IR ← instrução ADD
T4	~Lm, ~Ei	operando do IR	MAR ← endereço do dado
T5	~CE, ~Lb	RAM	B ← D (o dado)

T6	$\sim La, Eu$	ULA ($A + B$)	$A \leftarrow A + D$



A só é reescrito no posedge de T6 → o novo valor aparece já no T1 seguinte.

Figura 9 — Fluxo do ADD estado a estado: quem fala e quem ouve no barramento W.

A figura traduz a tabela acima em transferências pelo barramento: em cada estado, uma fonte (à esquerda) escreve no barramento W e um destino (à direita) o amostra. Vê-se que a busca (T1-T3, em azul) é idêntica à de qualquer instrução e que a identidade do ADD só aparece na execução (T4-T6, em verde), culminando em $A \leftarrow A + B$.

Observações. Em T1-T3 (busca) o processador nem sabe qual é a instrução — só em T3 o IR é carregado e o opcode passa a valer. Em T4 o operando (que é um **endereço**) é levado ao MAR; em T5 o dado lido vai para B; em T6 a ULA soma $A + B$ e o resultado volta para A. Como A só é escrito no posedge de T6, seu novo valor aparece "assentado" já no T1 da próxima instrução — exatamente o efeito de "um passo depois" visível nas ondas.

As outras instruções são variações deste esqueleto: LDA substitui o T5 por $A \leftarrow RAM$ (e não usa T6); SUB é igual ao ADD com $Su = 1$ em T6; OUT resolve em T4 (saída $\leftarrow A$) e deixa T5/T6 vazios; HLT apenas trava em T4.

9. Camada de placa (fpga/)

A pasta fpga adapta o núcleo à DE10-Lite sem tocar no rtl. O top de placa (sap1_fpga.v) reúne quatro cuidados de integração.

9.1. Geração e seleção de clock

O clock de 50 MHz é rápido demais para observar a execução. Um divisor gera ~1 Hz (um estado T por segundo), e um seletor permite alternar para clock manual passo a passo (um pulso por toque de botão), ideal para depurar e gravar vídeo:

rtl/./fpga/sap1_fpga.v – clocks

```
clock_divider #(.DIV(DIV)) u_div (.clk_in(clk50), .rst_n(rst_n), .clk_out(slow_clk));
debouncer     #(.N(500_000)) u_db (.clk(clk50), .rst_n(rst_n),
                                   .noisy(~KEY[1]), .clean(step_clk));
wire cpu_clk = SW[0] ? step_clk : slow_clk; // SW[0]: 0=auto(1Hz) 1>manual
```

O divisor conta $DIV-1$ e alterna a saída ($\text{freq} = 50 \text{ MHz} / (2 \cdot \text{DIV})$). O debouncer filtra o botão com um sincronizador de dois estágios seguido de um contador: só aceita o novo nível após ele permanecer estável por N ciclos (~10 ms), evitando múltiplos pulsos por toque.

9.2. Sincronizador de reset

O botão de reset é assíncrono ao clock. Para evitar violações de recovery/removal, o reset é aplicado de forma assíncrona (assert imediato) mas liberado de forma síncrona (desassert passa por dois flip-flops). Isso, somado ao estado IDLE do núcleo (seção 5.6), torna a partida totalmente determinística:

fpga/sap1_fpga.v – sincronizador de reset

```
always @(posedge clk50 or negedge ext_rst_n) begin
    if (!ext_rst_n) begin rs0 <= 0; rs1 <= 0; end // assert assíncrono
    else                begin rs0 <= 1; rs1 <= rs0; end // desassert sincronizado
end
wire rst_n = rs1; // ~CLR limpo para o núcleo
```

9.3. Mapeamento dos displays

Os seis displays de 7 segmentos mostram o funcionamento em tempo real. O opcode é traduzido para uma letra (em vez do hex cru) por um decodificador dedicado:

Display	Mostra	Fonte
HEX5	estado T atual (1-6; 0 = idle)	tstate → número
HEX4	instrução como letra: L A 5 o H	seg7_instr(opcode)
HEX3-HEX2	acumulador A (hex)	acc (registrador real)
HEX1-HEX0	resultado (registrador de	out_port

	saída, hex)	
LEDR[9]	HLT (aceso ao parar)	hlt

fpga/seg7_instr.v – opcode → letra (segmentos ativos em baixo)

```
always @(*) case (opcode)
  4'b0000: seg = 7'b1000111; // L  (LDA)
  4'b0001: seg = 7'b0001000; // A  (ADD)
  4'b0010: seg = 7'b0010010; // 5  (SUB, ~ 'S')
  4'b1110: seg = 7'b0100011; // o  (OUT)
  4'b1111: seg = 7'b0001001; // H  (HLT)
  default: seg = 7'b1111111; // apagado
endcase
```

Há ainda um modo de depuração (SW[1] = 1) que redireciona os displays para mostrar o registrador B e o próprio barramento W a cada estado — útil para acompanhar as transferências internas na placa.

10. Código-fonte comentado dos módulos-chave

Antes do código, esta seção fixa os conceitos de Verilog necessários para lê-lo; em seguida, cada módulo é apresentado com o código e uma explicação das construções usadas.

10.1. Conceitos de Verilog usados

Verilog descreve hardware, não um programa que executa passo a passo. Dois estilos convivem: lógica combinacional (a saída segue as entradas continuamente, como um circuito de portas) e lógica sequencial (o valor só muda na borda do clock, formando registradores). As construções abaixo são as usadas neste projeto:

Construção	O que significa
module ... (portas)	define um bloco de hardware e suas conexões externas
input / output	direção de cada porta do módulo
wire	fio: valor contínuo, dirigido por assign ou por outro bloco; não guarda estado
reg	sinal atribuído dentro de um bloco always; vira registrador (se sequencial) ou lógica (se combinacional)
assign x = ...	atribuição contínua ⇒ lógica combinacional (o fio acompanha a expressão o tempo todo)
always @(*)	bloco combinacional: reavalia quando qualquer entrada muda

always @(posedge clk)	bloco sequencial: age na borda de subida do clock ⇒ gera flip-flops
... or negedge clr_bar	acrescenta o reset à lista de sensibilidade ⇒ reset assíncrono
<= (não-bloqueante)	usado na lógica sequencial; as atribuições do bloco valem "todas juntas" na borda
= (bloqueante)	usado na lógica combinacional (always @(*))
localparam	constante nomeada (ex.: opcodes, códigos de estado)
case	seleção múltipla — usada para decodificar opcode/estado
{a, b}	concatenação de bits (ex.: estender 4→8 bits)
x[7:4]	fatiamento: seleciona um intervalo de bits
cond ? a : b	operador ternário — um multiplexador de duas vias

Reset assíncrono: quando `clr_bar` aparece na lista de sensibilidade (`negedge clr_bar`), o registrador zera no instante em que o reset é ativado, sem esperar o clock — por isso o `if (!clr_bar)` vem sempre primeiro (prioridade).

Por que <= (não-bloqueante): na lógica sequencial, todas as atribuições de um bloco devem enxergar os valores antigos e mudar simultaneamente na borda. O `<=` garante isso e evita corridas; o `=` (bloqueante) fica reservado à lógica combinacional.

reg não é "registrador" por si só: `reg` apenas indica que o sinal é atribuído dentro de um `always`. Se o `always` for `@(posedge clk)`, a síntese cria um flip-flop; se for `@(*)`, cria lógica combinacional.

10.2. Program Counter

`rtl/program_counter.v`

```
module program_counter (
    input wire    clk,
    input wire    clr_bar,    // ~CLR
    input wire    cp,        // Cp: habilita contagem
    output reg [3:0] pc_out
);
    always @(posedge clk or negedge clr_bar) begin
        if (!clr_bar)    pc_out <= 4'b0000;
        else if (cp)     pc_out <= pc_out + 4'b0001;
    end
endmodule
```

Leitura do código: `pc_out` é output reg [3:0] — quatro bits (0 a 15) atribuídos dentro do `always`, logo é um registrador. O bloco reage a duas bordas: a

subida do clock e a descida de `clr_bar`. A ordem das condições define a prioridade: primeiro `if (!clr_bar)` zera (reset assíncrono); senão, na borda de clock, se `cp = 1`, soma 1 (contagem); se `cp = 0`, não há atribuição e o valor é mantido — a síntese infere um flip-flop com enable. Ao passar de 15, os 4 bits voltam a 0 naturalmente (wraparound).

10.3. Registrador de Instrução (decodifica opcode | operando)

`rtl/instruction_register.v`

```
module instruction_register (
    input wire clk, clr_bar, li_bar,
    input wire [7:0] bus_in,
    output wire [3:0] opcode,    // nibble alto
    output wire [3:0] operand    // nibble baixo
);
    reg [7:0] ir;
    always @(posedge clk or negedge clr_bar) begin
        if (!clr_bar) ir <= 8'b0;
        else if (!li_bar) ir <= bus_in;
    end
    assign opcode = ir[7:4];
    assign operand = ir[3:0];
endmodule
```

Leitura do código: o registrador interno `ir` (8 bits) segue o mesmo padrão do PC — carrega `bus_in` quando `li_bar = 0`, senão mantém. A parte nova são os dois `assign`: como são atribuições contínuas, `opcode` e `operand` são `wire` e refletem sempre o conteúdo de `ir`. O fatiamento `ir[7:4]` pega os quatro bits altos (a instrução) e `ir[3:0]` os quatro baixos (o operando/endereço). Ou seja, a "decodificação" em dois campos não custa nenhum circuito extra — é apenas fiação para as duas metades do registrador.

MAR, registrador B e registrador de saída seguem exatamente o padrão do acumulador (seção 5.2), mudando apenas a largura (MAR tem 4 bits) e o nome da carga.

10.4. Integração no topo (instanciação)

`rtl/sap1_top.v` – trecho de instanciação

```
wire clk_cpu = clk;    // clock unico, SEM gating (HLT usa clock-enable)

accumulator u_acc (.clk(clk_cpu), .clr_bar(clr_bar), .la_bar(la_bar),
    .bus_in(w_bus), .acc_out(acc_out));
adder_subtractor u_alu (.su(su), .acc(acc_out), .breg(b_out), .result(alu_out));
controller_sequencer u_ctrl (.clk(clk_cpu), .clr_bar(clr_bar), .opcode(ir_opcode),
    .cp(cp), .ep(ep), .lm_bar(lm_bar), .ce_bar(ce_bar), .li_bar(li_bar),
    .ei_bar(ei_bar), .la_bar(la_bar), .ea(ea), .su(su), .eu(eu),
    .lb_bar(lb_bar), .lo_bar(lo_bar), .con(con), .ring(ring), .hlt(hlt));
```

Leitura do código: instanciar um módulo é criar uma cópia física do bloco e ligar seus pinos. A forma `nome_do_modulo nome_da_instancia (.porta(sinal), ...)` faz a conexão por nome — por exemplo, `.bus_in(w_bus)` liga a entrada `bus_in` do acumulador ao barramento. O mesmo `w_bus` chega a vários blocos

ao mesmo tempo (todos podem ler), mas só um escreve por vez, graças ao mux. Note que `clk_cpu` é apenas o clock de entrada sem nenhuma porta lógica no caminho — coerente com a decisão de não fazer gating (o HLT congela via clock-enable, não desligando o clock).

11. Execução completa de um programa (passo a passo)

Para fechar o funcionamento, esta seção acompanha um programa inteiro do início ao fim, amarrando o que foi visto em separado — estados, palavra de controle e datapath. Usamos o programa da multiplicação $3 \times 4 = 12$ (somas repetidas), por ser curto e visualmente claro. A memória contém, nos dados, `mem[11] = 3` e `mem[12..15] = 0`.

A tabela mostra cada instrução com a ação executada (resumida pelos estados usados) e os valores dos registradores após terminá-la. Cada instrução consome os seis estados T1-T6; a busca (T1-T3) é sempre a mesma e foi omitida da coluna de ação por brevidade.

Endereço	Instrução	Ação (execução)	A após	Saída
0	LDA 11	T4 MAR←11; T5 A←mem[11]=3	3	0
1	ADD 11	T4 MAR←11; T5 B←3; T6 A←A+B	6	0
2	ADD 11	T5 B←3; T6 A←A+B	9	0
3	ADD 11	T5 B←3; T6 A←A+B	12	0
4	OUT	T4 saída←A	12	12
5	SUB 12	T5 B←mem[12]=0; T6 A←A-B	12	12
6	ADD 13	T5 B←0; T6 A←A+B	12	12
7	SUB 14	T5 B←0; T6 A←A-B	12	12
8	ADD 15	T5 B←0; T6 A←A+B	12	12
9	OUT	T4 saída←A	12	12
10	HLT	T4 trava o anel (halted←1)	12	12

Lendo a tabela: as quatro primeiras instruções constroem a multiplicação somando o 3 repetidamente ($A: 3 \rightarrow 6 \rightarrow 9 \rightarrow 12$) — é assim que o SAP-1 "multiplica", já que não há instrução de multiplicação nem laços. Em seguida, o OUT (endereço 4) copia 12 para o visor. As instruções 5 a 8 operam com dados iguais a zero, de modo que A não muda — servem apenas para preencher o programa até o segundo OUT, que confirma o resultado. Por fim, o HLT congela a máquina em T4, mantendo 12 no visor. Esse é exatamente o comportamento observado nas formas de onda da Parte II.

11.1. As demais instruções, em resumo

O ADD e o SUB já foram rastreados ciclo a ciclo na seção 8. As outras três, que usam menos estados, completam o quadro:

LDA end — carrega da memória. Após a busca, no T4 o operando vai ao MAR e no T5 o dado lido da RAM é escrito no acumulador ($A \leftarrow \text{mem}[\text{end}]$). Termina em T5 (o T6 fica vazio). É a única forma de trazer um valor da memória para dentro do processador.

OUT — exhibe o acumulador. Após a busca, resolve-se tudo no T4: o acumulador escreve no barramento (Ea) e o registrador de saída o captura ($\sim\text{Lo}$). Não altera A nem a memória — apenas mostra o valor no visor. T5 e T6 ficam vazios.

HLT — para o processador. Após a busca, no T4 o flag de parada é travado ($\text{halted} \leftarrow 1$) e o contador de anel deixa de avançar (congela em T4). Todos os registradores mantêm seus valores e o clock continua correndo; só um reset reinicia a execução.

12. Conclusão

A implementação seguiu três eixos. Modularidade: um arquivo por bloco, com o topo apenas interligando-os pelo barramento, e uma separação estrita rtl × fpga. Boas práticas de FPGA no nível de hardware: barramento por multiplexador (sem tri-state), reset assíncrono com sincronizador de desassert, ausência total de gating de clock (HLT por clock-enable) e disciplina de duas bordas para eliminar corrida entre controle e dados. Clareza didática: palavra de controle transcrita da tabela de microoperações, máquina de estados explícita e disponível em duas formas equivalentes (anel one-hot e FSM clássica). O resultado é um núcleo sintetizável, determinístico na partida e legível — cuja unidade de controle, o ponto mais delicado, está descrita de forma direta e verificável. A validação por formas de onda consta do relatório de verificação por simulação.

Parte II — Verificação por simulação

Esta parte descreve a metodologia de simulação, a verificação de cada componente por formas de onda e a execução dos quatro programas de teste. A numeração de seções a seguir é interna a esta parte.

1. Introdução

Este relatório documenta a verificação funcional do processador didático SAP-1 (Simple-As-Possible), implementado em Verilog e validado por simulação antes da gravação na FPGA. O foco não é a arquitetura em si, mas a comprovação, por formas de onda e testes auto-verificáveis, de que cada componente e o sistema completo se comportam conforme o especificado.

Características relevantes do processador para a verificação:

- **Barramento único de 8 bits** (barramento W), implementado por multiplexador.
- **Memória RAM 16×8** (16 posições de 8 bits), somente leitura em execução, compartilhada entre programa e dados (arquitetura de Von Neumann).
- **Cinco instruções:** LDA (0000), ADD (0001), SUB (0010), OUT (1110) e HLT (1111); cada palavra é [opcode(4) | operando(4)], com endereçamento direto.
- **Unidade de controle** baseada em máquina de estados (contador de anel) com seis estados de temporização T1–T6: T1–T3 realizam a busca (iguais para toda instrução) e T4–T6 a execução (dependente do opcode).
- **Palavra de controle de 12 bits** (CON), gerada a cada estado, que comanda quem escreve e quem lê no barramento.

2. Metodologia de simulação

2.1. Ferramentas

A verificação foi conduzida principalmente no ModelSim ASE 2020.1 (Intel FPGA Starter Edition), o simulador integrado ao Quartus Prime Lite. As formas de onda apresentadas neste relatório foram capturadas diretamente da janela Wave do ModelSim. Como referência cruzada, os mesmos testbenches também foram executados em Icarus Verilog, obtendo-se resultados idênticos.

2.2. Testbenches auto-verificáveis

Foram escritos 14 testbenches — um para cada componente e um para o sistema integrado. Todos são auto-verificáveis: comparam o valor obtido com

o esperado e emitem PASSOU ou FALHOU no transcript, encerrando com \$stop. Não geram arquivos de dump (VCD), seguindo o fluxo nativo do ModelSim, no qual os sinais são observados diretamente na base de dados de simulação (WLF).

2.3. Modos de simulação empregados

A verificação usou quatro modos complementares:

- **Simulação unitária (por componente):** cada bloco é compilado e exercitado isoladamente pelo seu testbench, permitindo isolar falhas. Executada por componente com o script parametrizado run_one.do.
- **Simulação integrada (sistema completo):** o testbench tb_sap1 instancia o processador inteiro, carrega um programa na RAM e verifica a saída final (instrução OUT) e a parada correta em HLT.
- **Formas de onda didáticas:** scripts wave_*.do configuram a janela Wave com radix personalizados — o estado do anel aparece como T1...T6 e o opcode como mnemônico (LDA, ADD, ...) — e os registradores em decimal, facilitando a leitura.
- **Execução em lote:** o script run_all_tb.do compila e roda os 14 testbenches em sequência, consolidando os resultados PASSOU/FALHOU.

Radix personalizados definidos nos scripts de onda (trecho):

```
radix define ESTADO {  
    6'b000001 "T1", 6'b000010 "T2", 6'b000100 "T3",  
    6'b001000 "T4", 6'b010000 "T5", 6'b100000 "T6" }  
radix define OPCODE {  
    4'b0000 "LDA", 4'b0001 "ADD", 4'b0010 "SUB",  
    4'b1110 "OUT", 4'b1111 "HLT" }
```

Comandos típicos de reprodução (dentro da pasta tb_model):

```
# simulacao integrada com ondas do sistema:  
vsim -do wave_sap1.do  
# ondas de um componente (ex.: contador de programa):  
vsim -do wave_pc.do  
# lote com todos os testbenches:  
vsim -c -do run_all_tb.do
```

2.4. Testbenches implementados

Testbench	Alvo	Tipo de verificação
tb_program_counter	Program Counter	contagem, reset, pausa
tb_mar	MAR	carga/retenção de endereço
tb_ram_16x8	RAM 16×8	leitura do conteúdo por endereço
tb_instruction_register	Registrador de instrução	separação opcode

		operando
tb_accumulator	Acumulador (A)	carga/retenção
tb_register_b	Registrador B	carga/retenção
tb_adder_subtractor	ULA	soma, subtração, wraparound 8 bits
tb_output_register	Registrador de saída	carga/retenção
tb_controller_sequence r	Controlador/Sequenciador	estados e palavra de controle
tb_seg7 / tb_seg7_instr	Decodificadores 7-seg	mapeamento de dígitos/letras
tb_clock_divider / tb_debouncer	Apoio de FPGA	divisão de clock / antirruído
tb_sap1	Sistema completo	execução de programa fim-a-fim

3. Verificação dos componentes

Esta seção apresenta as formas de onda da simulação unitária dos principais componentes. Um padrão de projeto recorrente é o registrador com carga ativa em baixo: o bloco captura o barramento apenas quando seu sinal de carga vale 0 (na borda de subida do clock) e mantém o valor caso contrário; o reset assíncrono zera a saída. Esse padrão é compartilhado por MAR, IR, acumulador, registrador B e registrador de saída.

3.1. Program Counter (PC)

O que se espera: um contador de 4 bits que (i) incrementa quando habilitado, (ii) zera no reset e (iii) congela quando desabilitado. Por ser de 4 bits, deve contar de 0 a 15 e dar a volta (aritmética módulo 16), o que casa com as 16 posições da memória — o PC nunca aponta para um endereço inexistente.

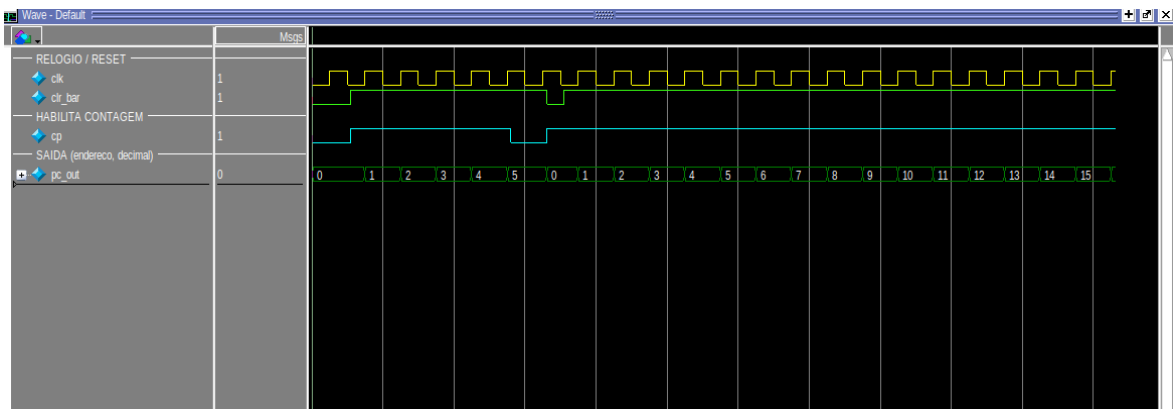


Figura 1 — Contador de programa: contagem, reset e pausa.

A onda comprova as três operações. Com o sinal de habilitação $cp = 1$, a saída avança 0, 1, 2, 3, ... a cada borda de subida do clock — exatamente um incremento por ciclo, sem saltos. Quando clr_bar é levado a 0, o contador retorna a 0 imediatamente, sem esperar o clock (reset assíncrono), como esperado de um sinal de inicialização. Com $cp = 0$, a contagem é congelada e o valor permanece estável mesmo com o clock ativo — esse congelamento é fundamental: é o mesmo mecanismo usado para o PC não avançar fora dos momentos certos do ciclo de instrução. Por fim, ao chegar em 15 a saída retorna a 0, confirmando o wraparound de 4 bits.

3.2. Registrador de Endereço da Memória (MAR)

O que se espera: um registrador de 4 bits que capture o endereço presente no barramento apenas quando sua carga estiver ativa ($lm_bar = 0$) e o segure estável durante toda a leitura da RAM. O ponto crítico a verificar é a retenção: o MAR precisa manter o endereço mesmo depois que o barramento muda, senão a RAM leria a posição errada.

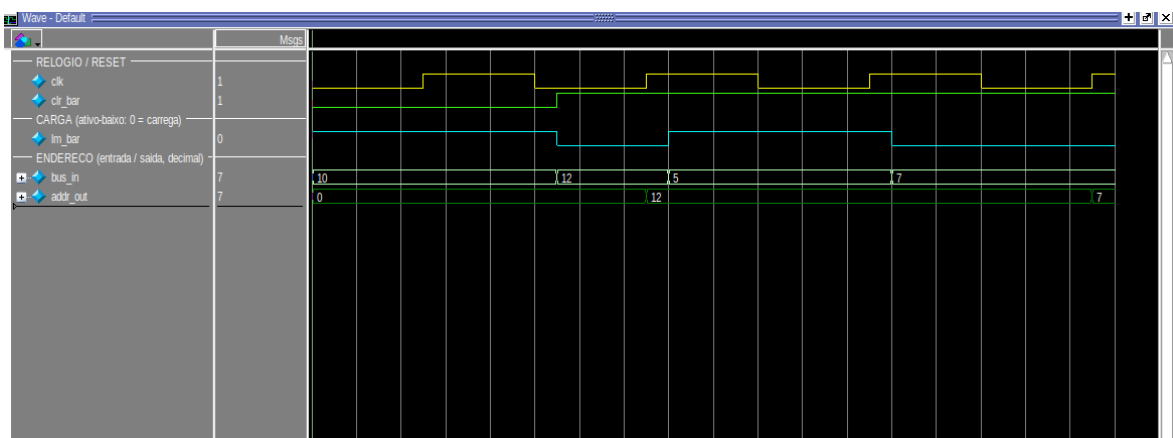


Figura 2 — MAR: carga de endereço e retenção.

A onda confirma o comportamento esperado. Com $lm_bar = 0$ o MAR carrega o valor da entrada (por exemplo, 12) na subida do clock. Em seguida, embora

a entrada mude para 5, a saída permanece em 12, pois Im_bar voltou a 1 — a retenção funciona. Só há nova atualização quando a carga é reativada (passando a 7). Esse endereço estável é justamente o que a RAM usa: durante uma instrução, o MAR é carregado duas vezes — no T1 com o endereço da instrução (vindo do PC) e no T4 com o operando (vindo do IR).

3.3. Memória RAM 16×8

O que se espera: uma memória de 16 palavras de 8 bits que devolva, para cada endereço, exatamente o byte gravado — sem clock, de forma combinacional. Espera-se ainda reconhecer, na palavra lida, os dois campos [opcode | operando] e confirmar que o operando é um endereço (endereçamento direto), não um valor imediato.

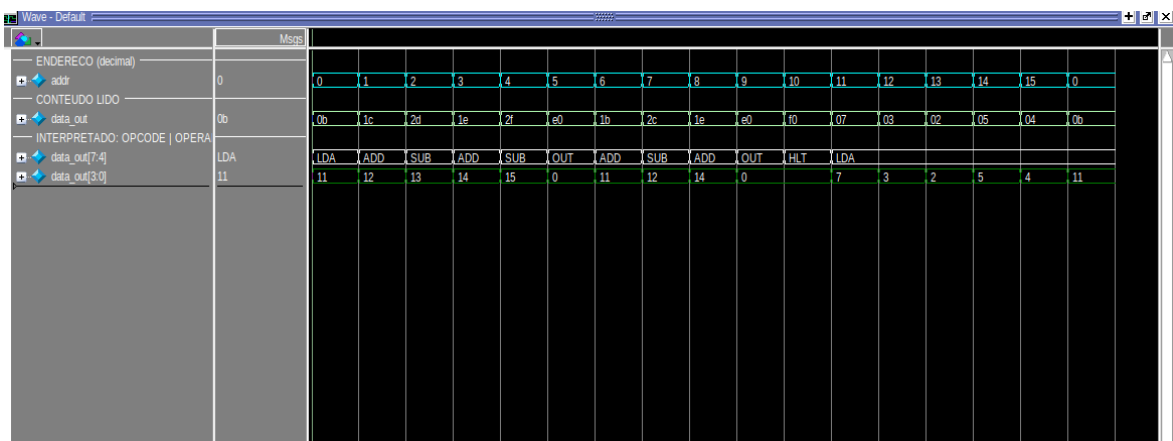


Figura 3 — RAM: leitura sequencial de todos os endereços (Programa 2).

O testbench percorre os 16 endereços e observa o conteúdo. A onda funciona como um "dump" do programa: endereço 0 → LDA 11, 1 → ADD 12, 2 → SUB 13, e assim por diante até HLT no endereço 10; os endereços 11 a 15 guardam os dados (7, 3, 2, 5, 4). Isso comprova, de forma visual, dois pontos conceituais importantes. Primeiro, o endereçamento direto: a linha interpretada mostra que o operando de "LDA 11" é o endereço 11 — para saber o valor, é preciso ir ler a posição 11 (que contém 7). Segundo, a coexistência de programa e dados na mesma memória (Von Neumann), o que explica por que os 16 slots precisam ser compartilhados. Como o SAP-1 básico não tem instrução de escrita (STA), essa memória é apenas de leitura em execução — na prática, uma ROM inicializada por um bloco inicial.

3.4. Registrador de Instrução (IR)

O que se espera: que a palavra de 8 bits vinda da memória seja capturada e dividida em dois campos de 4 bits — os bits altos formam o opcode (o que fazer) e os baixos, o operando (onde). É a etapa de "decodificação" do SAP-1.

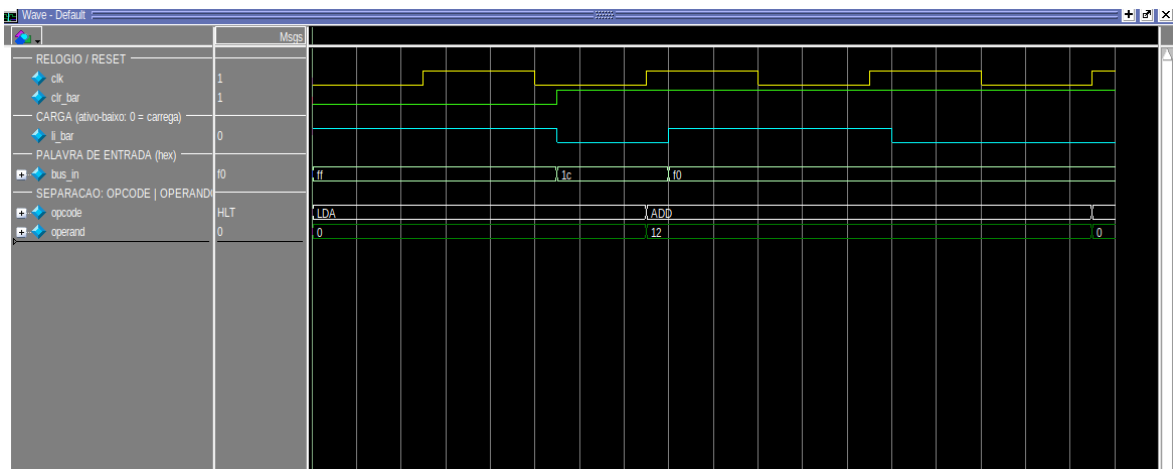


Figura 4 — IR: carga da palavra e separação opcode | operando.

A onda confirma a separação esperada. Ao carregar 0x1C (0001_1100) a saída se decompõe em opcode = ADD (0001) e operando = 12 (1100); ao carregar 0xF0 (1111_0000), em opcode = HLT e operando = 0. A partir daí os dois campos seguem caminhos diferentes: o opcode vai para a unidade de controle (que decide a sequência de microoperações) e o operando é colocado no barramento para virar o próximo endereço no MAR. Note que a saída só muda quando li_bar = 0, mantendo a instrução estável durante todos os estados de execução — a unidade de controle precisa do opcode fixo de T4 a T6.

3.5. Acumulador (A) e Registrador B

O que se espera: dois registradores de 8 bits idênticos em comportamento. O acumulador guarda o resultado corrente das contas; o B guarda o segundo operando da ULA. Espera-se, novamente, o padrão carrega-quando-habilitado / mantém-caso-contrário — essencial para que o valor de A sobreviva entre uma instrução e outra.

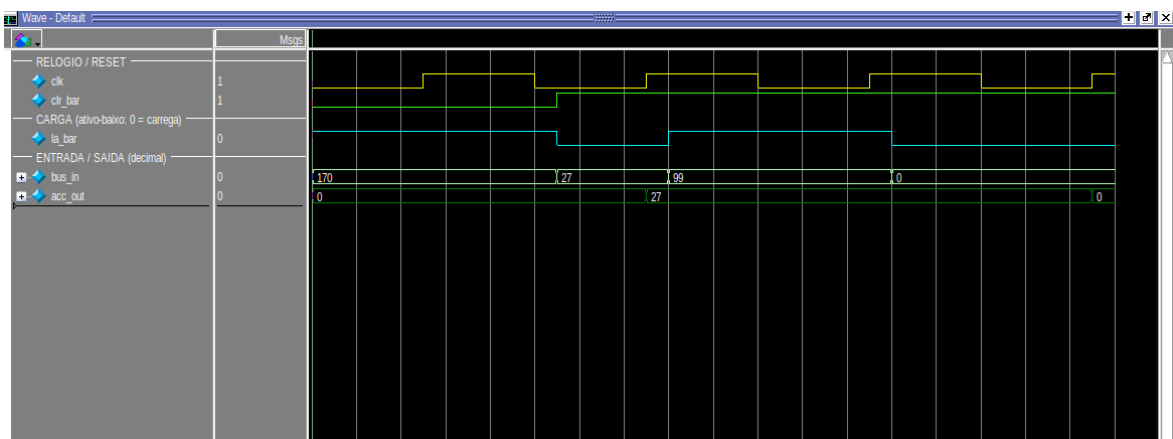


Figura 5 — Acumulador: carrega quando habilitado e mantém quando não.

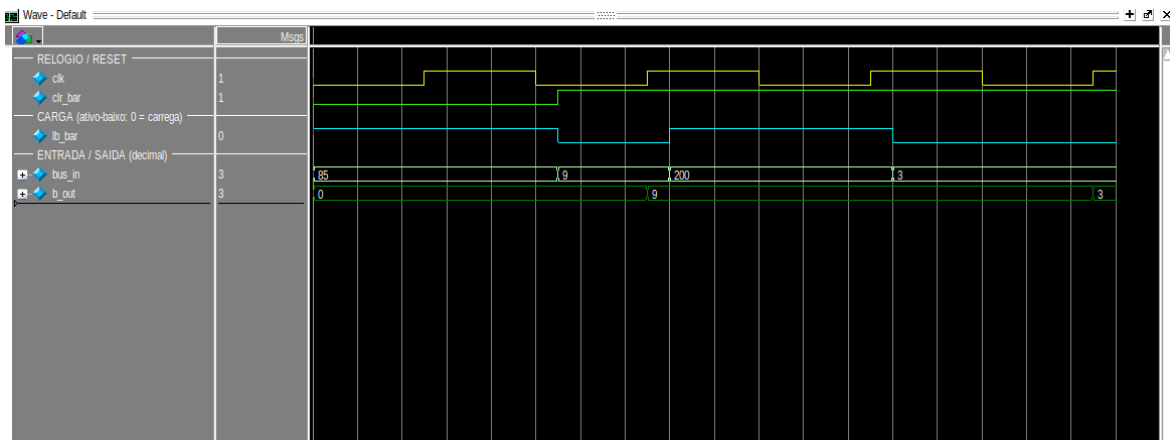


Figura 6 — Registrador B: mesmo padrão de carga/retenção.

As duas ondas confirmam o esperado. No acumulador, com $la_bar = 0$ a saída assume 27 e, mesmo com a entrada mudando para 99, permanece em 27 enquanto a carga não for reativada — depois carrega 0. O registrador B repete o padrão (carrega 9, mantém com a entrada em 200, depois carrega 3). Essa retenção é o que torna possível a aritmética acumulativa: em um ADD, o valor antigo de A é somado ao novo dado e o resultado volta para A. Vale destacar a diferença de papéis — enquanto o B é apenas um "copo" temporário para o segundo operando, o A é o registrador de trabalho, lido e reescrito repetidamente ao longo do programa.

3.6. Unidade Lógico-Aritmética (ULA)

O que se espera: um bloco puramente combinacional que calcule $A + B$ (quando $su = 0$) ou $A - B$ (quando $su = 1$), em 8 bits. Como não há bits extras, espera-se wraparound: somas acima de 255 e subtrações abaixo de 0 devem "dar a volta" (aritmética módulo 256 / complemento de dois).

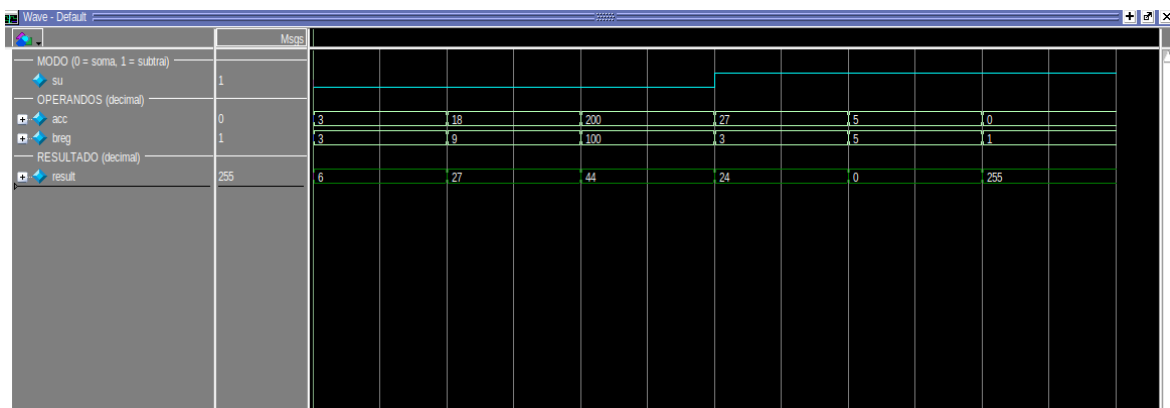


Figura 7 — ULA: soma, subtração e wraparound de 8 bits.

A onda valida todos os casos, inclusive os de borda. Com $su = 0$ (soma): $3+3 = 6$ e $18+9 = 27$, resultados diretos; e $200+100 = 44$, que é exatamente $300 - 256$ — o wraparound esperado. Com $su = 1$ (subtração): $27-3 = 24$, $5-5 =$

0 e, o caso mais instrutivo, $0-1 = 255$, que é a representação de -1 em complemento de dois com 8 bits. Por ser combinacional, o resultado acompanha as entradas sem atraso de clock; ele só chega ao barramento — e, portanto, ao acumulador — quando o sinal Eu é ativado (no estado T6 das instruções ADD/SUB). Esse limite de 8 bits é a razão de o SAP-1 só operar com valores de 0 a 255.

3.7. Controlador / Sequenciador

O que se espera: o bloco mais crítico do processador. Espera-se um contador de anel que percorra $IDLE \rightarrow T1 \rightarrow \dots \rightarrow T6 \rightarrow IDLE$ e, em cada estado, emita uma palavra de controle de 12 bits (CON) com a combinação exata de sinais daquele passo. A palavra deve depender apenas do estado e do opcode — nunca do dado — garantindo comportamento determinístico. Espera-se também que HLT trave a máquina em T4.

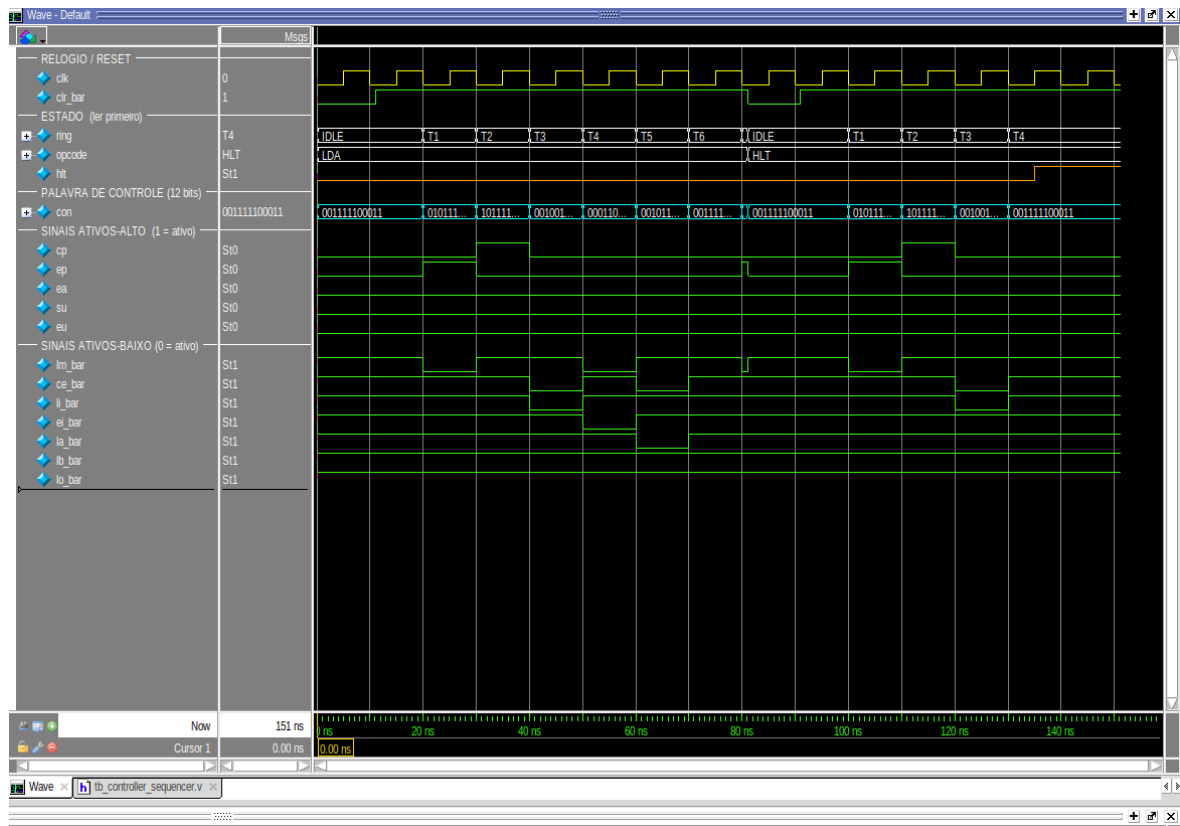


Figura 8 — Máquina de estados e palavra de controle de 12 bits por estado.

A onda confirma cada expectativa. O anel percorre $IDLE \rightarrow T1 \rightarrow T2 \rightarrow \dots \rightarrow T6 \rightarrow IDLE$, e a palavra CON assume um valor distinto e bem definido em cada estado. Os sinais ativos em alto (cp, ep, ea, su, eu) aparecem como pulsos nos estados corretos — por exemplo, ep no T1 (o PC fala) e cp no T2 (o PC incrementa) —, enquanto os sinais ativos em baixo (as cargas) descem a 0 apenas quando devem atuar. Um detalhe importante confirmado aqui: a

palavra de repouso não é toda-zero, justamente porque as cargas são ativas em baixo. Por fim, ao aplicar HLT a máquina permanece travada em T4, comprovando a parada correta. É desta tabela de palavras que nascem, coordenadamente, todos os movimentos vistos nas ondas anteriores.

4. Verificação do sistema completo

Com os componentes validados, o testbench `tb_sap1` exercita o processador inteiro. Para cada programa, a RAM é inicializada, a simulação roda até o HLT e a saída da instrução OUT é comparada, por assert automático, ao valor esperado. As ondas a seguir usam radix didáticos: o estado aparece como T1...T6, o opcode como mnemônico e o datapath (PC, MAR, A, B, ULA, saída) em decimal.

4.1. Modelo temporal esperado (como prever a onda)

Antes de analisar cada programa, convém fixar o modelo de tempo esperado — é ele que permite "prever" a onda e depois conferir. Duas regras de temporização governam tudo:

- O **anel de estados avança na borda de descida** do clock (negedge): é quando a máquina passa de T1 para T2, de T2 para T3, e assim por diante.
- Os **registradores carregam na borda de subida** (posedge), quando seu sinal de carga está ativo. Ou seja, o controle "prepara" o sinal em um estado e o dado é efetivamente capturado no flanco seguinte.

Com isso, cada instrução consome exatamente 6 estados (T1–T6). Os três primeiros são a busca, idênticos para toda instrução:

T1:	PC -> MAR	(endereço da instrução vai para o MAR)
T2:	PC = PC + 1	(aponta para a próxima instrução)
T3:	RAM -> IR	(a instrução é lida e decodificada)

Os três últimos (execução) dependem do opcode:

LDA:	T4 IR.op -> MAR	T5 RAM -> A	(A muda no T5)
ADD:	T4 IR.op -> MAR	T5 RAM -> B	T6 ULA(A+B) -> A
SUB:	T4 IR.op -> MAR	T5 RAM -> B	T6 ULA(A-B) -> A
OUT:	T4 A -> registrador de saída		
HLT:	T4 trava o anel	(congela em T4, sem parar o clock)	

Consequência prática na leitura da onda: em um ADD/SUB, o novo valor do acumulador só aparece no T6 e, portanto, torna-se visível como valor "assentado" já no T1 da instrução seguinte. Por isso, ao ler as ondas, o acumulador parece mudar "um passo depois" — não é atraso indevido, é o modelo esperado. Esse detalhe é a principal fonte de confusão ao interpretar formas de onda de processadores, e a simulação o torna concreto.

Número de ciclos esperado: como cada instrução leva 6 estados, um programa com N instruções (contando o HLT) deve parar em torno de $6 \times N$ ciclos. Usaremos essa conta para conferir cada caso.

4.2. Programa 1 — $3^3 = 27$

Potência por somas sucessivas; exibe 9 (3^2) e depois 27 (3^3).

Listagem (endereço : instrução):

0	LDA 12	; A <- 3	8	ADD 12	; A <- 27
1	ADD 12	; A <- 6	9	ADD 15	; A <- 27 (3^3)
2	ADD 12	; A <- 9 (3^2)	10	OUT	; mostra 27
3	OUT	; mostra 9	11	HLT	
4	LDA 13	; A <- 9	12	dado = 3	
5	ADD 13	; A <- 18	13	dado = 9	
6	ADD 13	; A <- 27	14	dado = 3	
7	SUB 14	; A <- 24	15	dado = 0	

Resultado esperado (dedução): $3^3 = 3 \times 3 \times 3 = 27$. Sem multiplicação nativa, o cálculo é feito em duas etapas: primeiro $3^2 = 3+3+3 = 9$ (exibido no primeiro OUT); depois $3^3 = 9 \times 3 = 9+9+9 = 27$. As instruções 7 e 8 (SUB 3 e ADD 3) se cancelam, servindo apenas para ajustar o acumulador ao valor de dado disponível — uma limitação típica de não ter registrador extra. O valor final esperado é 27 (0x1B).

Sequência esperada do acumulador: $3 \rightarrow 6 \rightarrow 9$ (OUT = 9) $\rightarrow 9 \rightarrow 18 \rightarrow 27 \rightarrow 24 \rightarrow 27 \rightarrow 27$ (OUT = 27).

Ciclos esperados: 12 instruções $\times$ 6 estados = 72 ciclos.

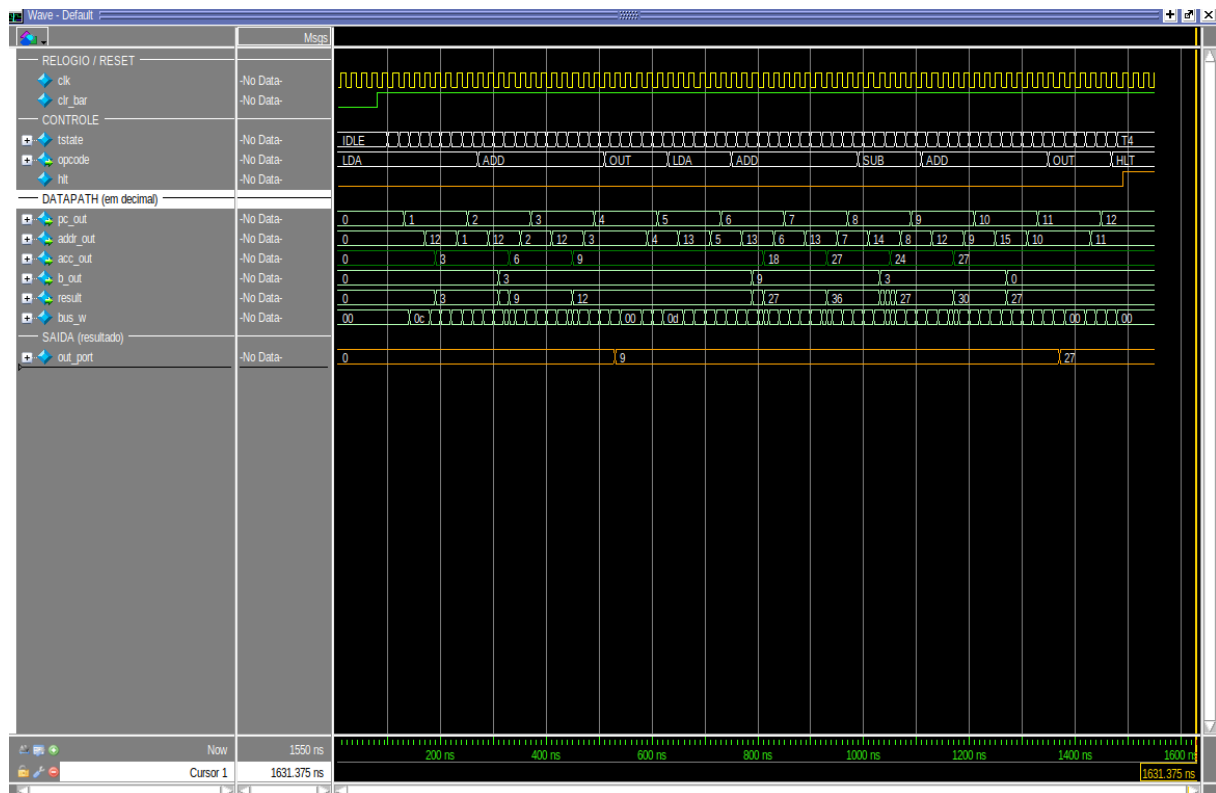


Figura 9 — 4.2. Programa 1 — $3^3 = 27$ — simulação completa.

O que a onda confirma: no acumulador (verde) vê-se exatamente 3, 6, 9 (primeiro OUT = 9) e, na segunda etapa, a subida até 27, com o segundo OUT exibindo 27. A saída (out_port) evolui $0 \rightarrow 9 \rightarrow 27$ e o processador para em HLT/T4. O assert do testbench confirma: obtido = esperado = 27.

4.3. Programa 2 — Expressão aritmética = 18

Cadeia de somas e subtrações: $((((7+3)-2)+5)-4, \text{ depois } +7-3+5; \text{ exibe } 9 \text{ e } 18)$.

Listagem (endereço : instrução):

0 LDA 11 ; A <- 7	6 ADD 11 ; +7 -> 16
1 ADD 12 ; +3 -> 10	7 SUB 12 ; -3 -> 13
2 SUB 13 ; -2 -> 8	8 ADD 14 ; +5 -> 18
3 ADD 14 ; +5 -> 13	9 OUT ; mostra 18
4 SUB 15 ; -4 -> 9	10 HLT
5 OUT ; mostra 9	11..15 dados = 7, 3, 2, 5, 4

Resultado esperado (dedução): A expressão avaliada é $((((7+3)-2)+5)-4 = 9$ no primeiro trecho e, continuando a partir de 9, $((((9+7)-3)+5) = 18$ no segundo. Cada operando é buscado por endereço na área de dados (posições 11 a 15). O valor final esperado é 18 (0x12).

Sequência esperada do acumulador: $7 \rightarrow 10 \rightarrow 8 \rightarrow 13 \rightarrow 9$ (OUT = 9) $\rightarrow 16 \rightarrow 13 \rightarrow 18$ (OUT = 18).

Ciclos esperados: 11 instruções × 6 estados = 66 ciclos (confirmado: HLT em 66 ciclos).

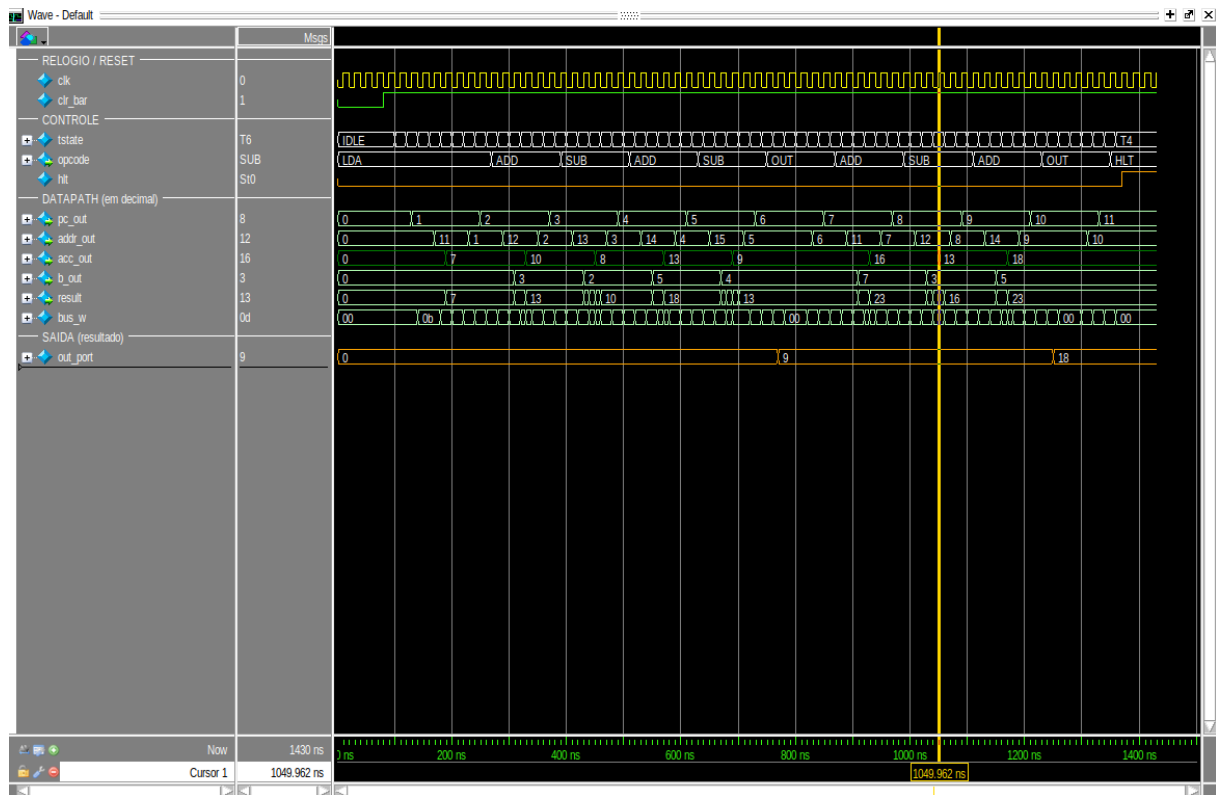


Figura 10 — 4.3. Programa 2 — Expressão aritmética = 18 — simulação completa.

O que a onda confirma: o acumulador percorre 7, 10, 8, 13, 9 e depois 16, 13, 18, batendo com a dedução termo a termo. Este é o programa que melhor mostra o encadeamento de operações e o uso de duas instruções OUT (resultados parciais). A saída evolui 0 → 9 → 18 e há parada em HLT/T4. Assert: obtido = esperado = 18.

4.4. Programa 3 — Multiplicação $3 \times 4 = 12$

Como não há instrução de multiplicação, faz-se por somas repetidas ($3+3+3+3$).

Listagem (endereço : instrução):

0 LDA 11 ; A <- 3	5 SUB 12 ; dado 0, A inalterado
1 ADD 11 ; A <- 6	6 ADD 13 ; dado 0, A inalterado
2 ADD 11 ; A <- 9	7 SUB 14 ; dado 0, A inalterado
3 ADD 11 ; A <- 12	8 ADD 15 ; dado 0, A inalterado
4 OUT ; mostra 12	9 OUT ; mostra 12
	10 HLT
	11 dado = 3

Resultado esperado (dedução): 3×4 significa somar o número 3 quatro vezes: $3+3+3+3 = 12$. Como o SAP-1 não tem laços (não há desvio), as quatro somas são escritas explicitamente. As instruções 5 a 8 operam com

dados iguais a 0, de modo a não alterar o acumulador — servem apenas para preencher o programa até o segundo OUT. O valor final esperado é 12 (0x0C).

Sequência esperada do acumulador: 3 → 6 → 9 → 12 (OUT = 12) → 12 → 12 → 12 (OUT = 12).

Ciclos esperados: 11 instruções × 6 estados = 66 ciclos (confirmado: HLT em 66 ciclos).

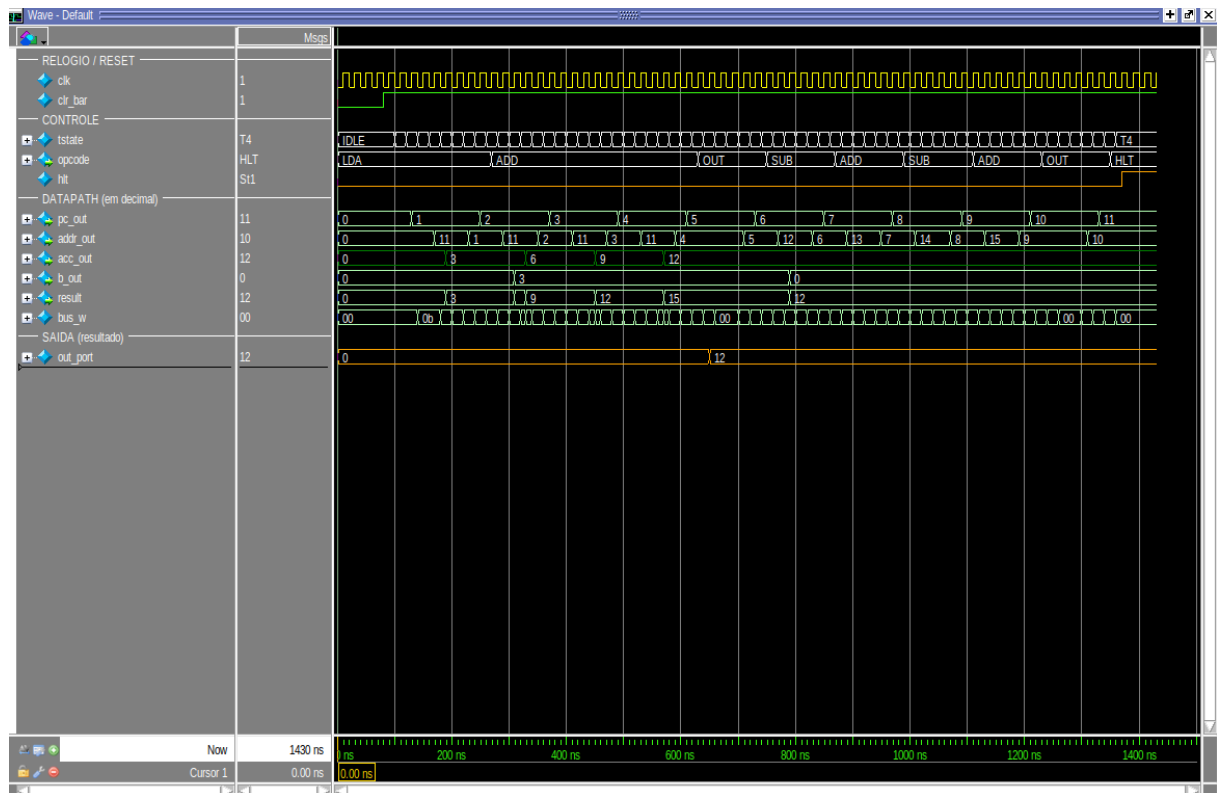


Figura 11 — 4.4. Programa 3 — Multiplicação $3 \times 4 = 12$ — simulação completa.

O que a onda confirma: a "assinatura" da multiplicação é inconfundível: o acumulador sobe de 3 em 3 (3, 6, 9, 12) e depois permanece constante (as somas/subtrações com 0). É a prova visual de que multiplicar, aqui, é repetir soma. A saída atinge 12 e há parada em HLT/T4. Assert: obtido = esperado = 12.

4.5. Programa 4 — Divisão $12 \div 4 = 3$

Divisão por subtrações repetidas: subtrai 4 até zerar e conta o quociente.

Listagem (endereço : instrução):

0 LDA 12 ; A <- 12	5 ADD 14 ; A <- 2
1 SUB 13 ; A <- 8	6 ADD 14 ; A <- 3 (quociente)
2 SUB 13 ; A <- 4	7 OUT ; mostra 3
3 SUB 13 ; A <- 0	8 HLT
4 LDA 14 ; A <- 1	12..14 dados = 12, 4, 1

Resultado esperado (dedução): $12 \div 4$ pergunta "quantas vezes o 4 cabe no 12?". A resposta é obtida subtraindo 4 repetidamente até zerar: $12 \rightarrow 8 \rightarrow 4 \rightarrow 0$, ou seja, 3 subtrações. Como não há como contar automaticamente (não há laço nem desvio), o quociente 3 é montado à mão, somando 1 três vezes. O número de subtrações é fixo para este caso específico — trocar os valores exigiria reescrever o programa. O valor final esperado é 3 (0x03).

Sequência esperada do acumulador: $12 \rightarrow 8 \rightarrow 4 \rightarrow 0$ (fim das subtrações) $\rightarrow 1 \rightarrow 2 \rightarrow 3$ (OUT = 3).

Ciclos esperados: 9 instruções $\times$ 6 estados = 54 ciclos (confirmado: HLT em 54 ciclos).

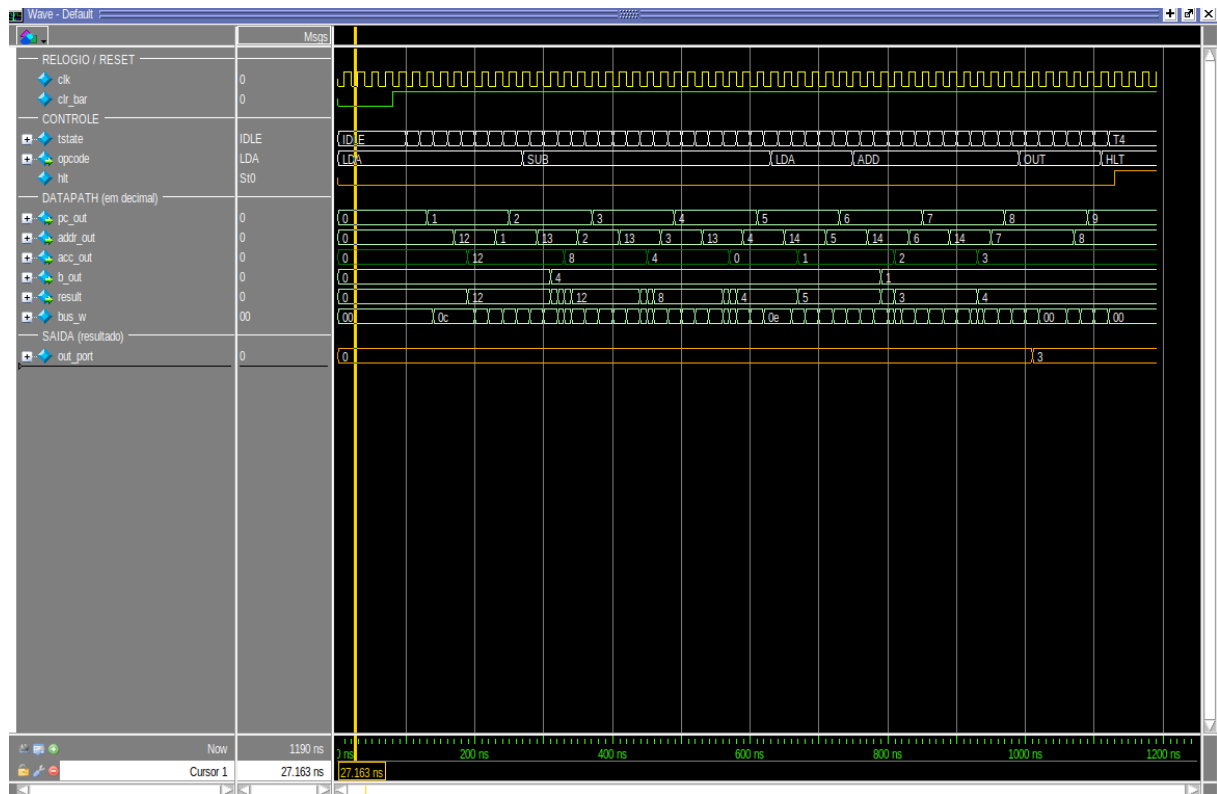


Figura 12 — 4.5. Programa 4 — Divisão $12 \div 4 = 3$ — simulação completa.

O que a onda confirma: a onda mostra o acumulador descendo de 4 em 4 (12, 8, 4, 0) — visualmente o oposto da multiplicação — e, em seguida, a contagem do quociente 1, 2, 3. A saída atinge 3 e há parada em HLT/T4. Assert: obtido = esperado = 3. Este caso evidencia bem a principal limitação do SAP-1: sem desvio condicional, o algoritmo não se adapta aos dados.

5. Resultados consolidados

Todos os 14 testbenches retornaram PASSOU, tanto no ModelSim ASE quanto no Icarus Verilog. Os quatro programas produziram exatamente a saída

esperada e pararam corretamente em HLT. A tabela abaixo resume a verificação do sistema completo.

Prog.	Operação	Técnica	Esper.	Obt.	Ciclos	Parada
1	3^3	somas sucessivas	27	27	72	HLT/T4
2	expressão	soma/subtração encadeadas	18	18	66	HLT/T4
3	3×4	somas repetidas	12	12	66	HLT/T4
4	$12 \div 4$	subtrações repetidas	3	3	54	HLT/T4

Resultado global: 14/14 testbenches PASSOU · 4/4 programas corretos.

6. Conclusão

A verificação por simulação confirmou o funcionamento correto do SAP-1 em dois níveis: os componentes, validados isoladamente por testbenches auto-verificáveis com formas de onda que evidenciam contagem, carga/retenção de registradores, leitura de memória e aritmética com wraparound; e o sistema completo, que executou os quatro programas produzindo os resultados esperados e parando corretamente em HLT. A combinação de simulação unitária e integrada, o uso de radix didáticos e a execução em lote deram confiança suficiente para prosseguir com a gravação na FPGA DE10-Lite. Como evolução natural, a inclusão de uma instrução de escrita (STA) e de desvios permitiria verificar laços e algoritmos com fluxo de controle, aproximando o projeto do SAP-2.

Execução na placa (DE10-Lite)

Além da simulação, os programas foram gravados e executados na FPGA DE10-Lite. Nos displays de 7 segmentos é possível acompanhar, em tempo real, o estado atual (T1-T6), a instrução (como letra), o acumulador e o resultado; o LED de HLT acende quando o processador para. As fotos abaixo registram cada programa em execução.

Instrução: substitua cada quadro cinza por uma foto do respectivo programa rodando na placa (clique no quadro e insira a imagem).

[espaço reservado para foto] Programa 1 — 3^3 resultado esperado no visor: 1B (27)	[espaço reservado para foto] Programa 2 — expressão resultado esperado no visor: 12 (18)
[espaço reservado para foto] Programa 3 — 3×4 resultado esperado no visor: 0C (12)	[espaço reservado para foto] Programa 4 — $12 \div 4$ resultado esperado no visor: 03 (3)